Университет ИТМО
Факультет программной инженерии и компьютерной техники

Курсовая работа

Этап 2. Модель данных и её реализация в PostgreSQL

Информационная система «Бдительность»

Выполнил: студент группы P3310
Мыц Иван Евгеньевич

Санкт-Петербург, 2026

Содержание

1 Задание **2**

2 ER-модель **2**

3 Даталогическая модель **4**

4 Реализация в PostgreSQL **7**

5 Обеспечение целостности данных **9**
5.1 Средства DDL 9
5.2 Триггеры 9
5.3 Проверка 11

6 Функции и процедуры **12**

7 Индексы **14**
7.1 Выбор индексов 14
7.2 Проверка эффективности 16

8 Заключение **17**

Приложение А. Листинги скриптов **18**
A.1. 01_create.sql 18
A.2. 02_triggers.sql 22
A.3. 03_functions.sql 31
A.4. 04_indexes.sql 38

1. Задание

1. Сформировать ER-модель базы данных на основе описаний предметной области и прецедентов из предыдущего этапа. ER-модель должна включать не менее 10 сущностей и содержать хотя бы одно отношение вида «многие-ко-многим».
2. Согласовать ER-модель с преподавателем. На основе ER-модели построить даталогическую модель.
3. Реализовать даталогическую модель в реляционной СУБД PostgreSQL.
4. Обеспечить целостность данных при помощи средств языка DDL и триггеров.
5. Реализовать скрипты для создания, удаления базы данных, заполнения базы тестовыми данными.
6. Предложить pl/pgsql-функции и процедуры для выполнения критически важных запросов, которые потребуются при последующей реализации прецедентов.
7. Создать индексы на основе анализа использования базы данных в контексте описанных на первом этапе прецедентов. Обосновать полезность созданных индексов для реализации представленных на первом этапе бизнес-процессов.
8. Составить отчёт.

Исходными данными являются спецификация требований (SRS) версии 1.0 и модель прецедентов UC01–UC23, разработанные на первом этапе. Ссылки на требования (U, C, S, SEC, REL и др.) и прецеденты (UC) в настоящем отчёте соответствуют этой спецификации.

2. ER-модель

ER-модель построена по описанию предметной области (раздел 2.1 SRS), словарю терминов (раздел 1.3 SRS) и спецификациям прецедентов (приложение A SRS). Модель представлена на рисунке 1 в нотации «вороньих лапок»: метка у конца связи обозначает кардинальность со стороны соответствующей сущности (две черты — ровно один, черта и кружок — ноль или один, «лапка» с кружком — ноль или много, «лапка» с чертой — один или много).

Модель содержит 16 сущностей. Связь «многие-ко-многим» между сущностями «Донос» и «Наказание» выделена на рисунке красным цветом. Статус благонадёжности жильца не является атрибутом: он однозначно определяется значением ИЛ и вычисляется при запросе. Диаграмма в исходном размере приложена к отчёту в файле `img/er.pdf`.

Таблица 1 – Сущности ER-модели

Сущность	**Назначение**	**Основные атрибуты**
Учётная запись	Данные для входа пользователя: жильца или управляющего.	логин, хеш пароля, роль, признак блокировки
Квартира	Жилое помещение дома, за которое начисляется квартплата.	номер, этаж, площадь, тариф
Жилец	Гражданин, проживающий в доме; хранит анкету, ИЛ и коммунальный баланс.	ФИО, дата рождения, телефон, ИЛ, баланс, статус
Код приглашения	Одноразовый код для активации учётной записи жильца.	код, даты выдачи и использования
Услуга	Услуга из фиксированного перечня (SV01–SV09).	код, название, минимальный ИЛ, пошлина, вид результата

Сущность	Назначение	Основные атрибуты
Заявка	Обращение жильца за услугой и его результат.	статус, пошлина, заключение, документ, срок действия
Событие заявки	Запись истории обработки заявки: смена статуса, вопрос или ответ.	исходный и новый статус, комментарий, дата
Донос	Сообщение жильца о нарушении, допущенном другим жильцом.	описание, анонимность, статус, решение
Наказание	Мера, назначенная жильцу: предупреждение, штраф, выселение.	вид, сумма, основание, признак снятия
Платёж	Пополнение баланса через ЮKassa.	идентификатор ЮKassa, сумма, статус
Начисление квартплаты	Ежемесячное начисление за квартиру.	период, сумма, срок оплаты, статус
Операция по балансу	Запись журнала изменений коммунального баланса.	тип, сумма, баланс после операции
Правило ИЛ	Правило изменения индекса лояльности (RL01–RL07).	код, описание, величина изменения
Изменение ИЛ	Запись журнала изменений индекса лояльности.	применённое изменение, значение после
Заметка досье	Наблюдение управляющего о жильце.	текст, дата
Уведомление	Сообщение пользователю о событии в системе.	текст, признак прочтения

Таблица 2 – Связи ER-модели

Связь	Кардинальность	Смысл
Учётная запись — Жилец	0..1 : 0..1	Жилец получает учётную запись после активации; у управляющего записи жильца нет.
Квартира — Жилец	1 : 0..*	Жилец проживает в квартире; в квартире одновременно проживает не более одного жильца, выселенные жильцы сохраняют связь с последней квартирой.
Жилец — Код приглашения	1 : 1	При заселении жильцу выдаётся код приглашения.
Жилец — Заявка	1 : 0..*	Жилец подаёт заявки.
Услуга — Заявка	1 : 0..*	Заявка подаётся на одну услугу.
Квартира — Заявка	0..1 : 0..*	Заявка на переселение (SV07) указывает целевую квартиру.
Заявка — Событие заявки	1 : 1..*	Заявка имеет историю обработки.
Учётная запись — Событие заявки	0..1 : 0..*	Пользователь, изменивший статус заявки.
Жилец — Донос	1 : 0..*	Жилец является автором доноса.
Жилец — Донос	1 : 0..*	Жилец является подозреваемым по доносу.
Учётная запись — Донос	0..1 : 0..*	Управляющий рассматривает донос.

Связь	Кардинальность	Смысл
Жилец — Наказание	1 : 0..*	Жилец получает наказание.
Учётная запись — Наказание	1 : 0..*	Управляющий назначает наказание.
Донос — Наказание	**0..* : 0..***	**Наказание назначается по одному или нескольким подтверждённым доносам об одном нарушении; по одному доносу может быть назначено несколько наказаний (например, штраф и предупреждение).**
Заявка — Наказание	0..1 : 0..1	Выполненная заявка SV09 снимает предупреждение.
Жилец — Платёж	1 : 0..*	Жилец пополняет баланс.
Жилец — Начисление квартплаты	1 : 0..*	Квартплата начисляется жильцу.
Квартира — Начисление квартплаты	1 : 0..*	Квартплата начисляется за квартиру.
Жилец — Операция по балансу	1 : 0..*	Операция изменяет баланс жильца.
Платёж, Начисление, Заявка, Наказание — Операция по балансу	0..1 : 0..1 (для заявки 0..1 : 0..*)	Источник операции: пополнение, оплата квартплаты, пошлина или её возврат, штраф.
Жилец — Изменение ИЛ	1 : 0..*	Изменение относится к жильцу.
Правило ИЛ — Изменение ИЛ	1 : 0..*	Изменение выполняется по правилу.
Донос, Наказание, Начисление, Заявка — Изменение ИЛ	0..1 : 0..*	Причина изменения ИЛ.
Жилец — Заметка досье	1 : 0..*	Заметка описывает жильца.
Учётная запись — Заметка досье	1 : 0..*	Управляющий составляет заметку.
Учётная запись — Уведомление	1 : 0..*	Пользователь получает уведомления.

Сущности «Операция по балансу» и «Изменение ИЛ» выделены как журналы: текущие значения баланса и ИЛ жильца всегда равны сумме записей журнала, а сами записи не изменяются (требование S013 SRS). Это позволяет восстановить историю изменений (UC05, UC12) и проверить корректность текущих значений.

3. Даталогическая модель

Даталогическая модель (рисунок 2) получена из ER-модели по следующим правилам.

1. Каждая сущность преобразована в таблицу с суррогатным первичным ключом `id`. Для справочника правил ИЛ используется естественный ключ `code`.
2. Связи «один-ко-многим» реализованы внешними ключами на стороне «многие».
3. Связи «один-к-одному» реализованы внешним ключом с ограничением `UNIQUE` (столбец `account_id` таблицы `resident`, столбец `resident_id` таблицы `invitation`, столбец `payment_id` таблицы `balance_operation` и др.).

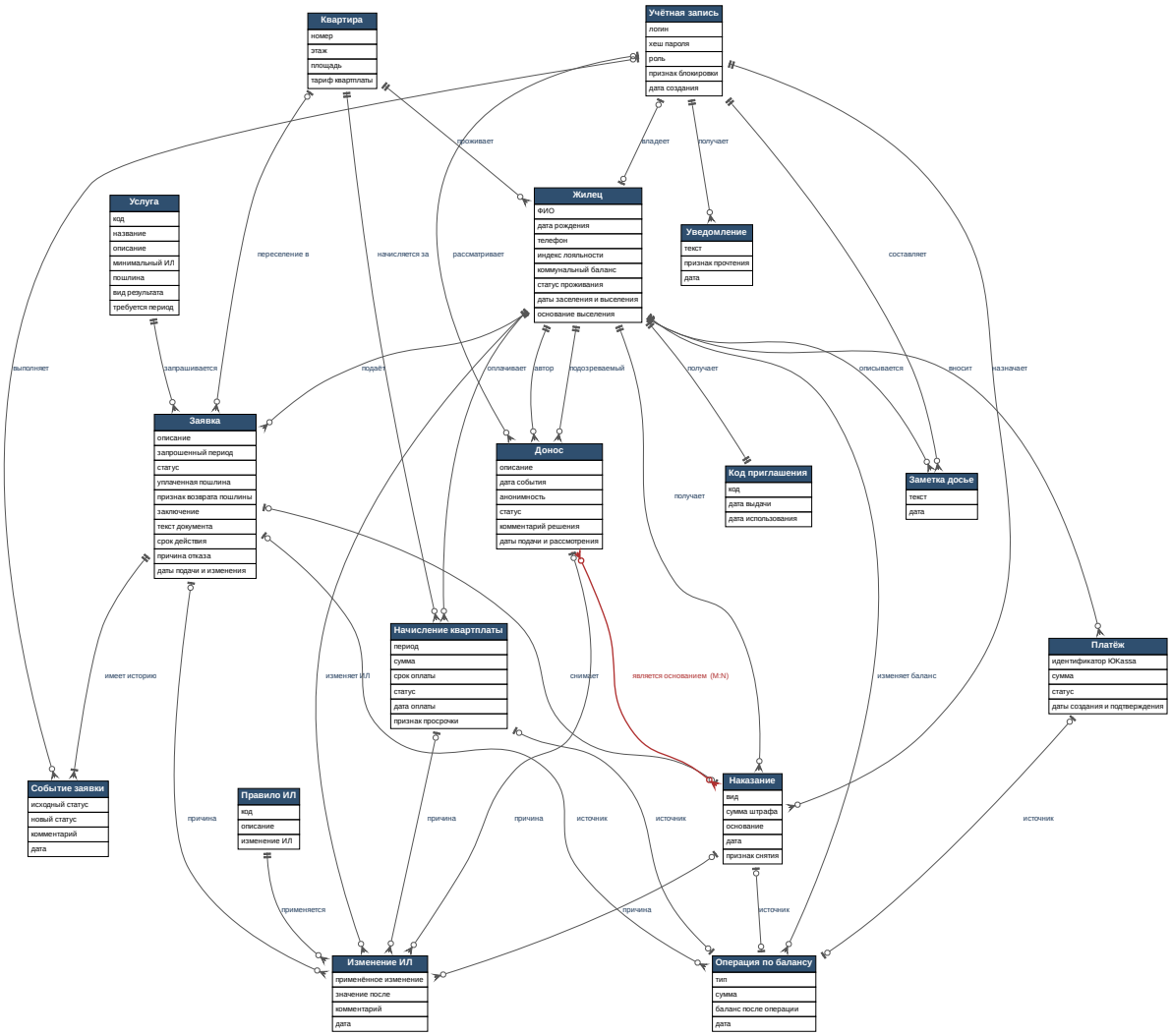


Рисунок 1 – ER-модель базы данных

4. Связь «многие-ко-многим» «Донос — Наказание» реализована ассоциативной таблицей `punishment_basis` с составным первичным ключом.
5. Атрибуты с фиксированным набором значений (роль, статусы, виды наказаний, типы операций) реализованы перечислимыми типами PostgreSQL (`CREATE TYPE … AS ENUM`).
6. Статус благонадёжности не хранится, а вычисляется по ИЛ функцией `reliability_status` (пороги из таблицы статусов SRS, требование S011).
7. Задолженность хранится как отрицательное значение `resident.balance`: при пополнении баланса задолженность погашается автоматически (требование S004 SRS).

Модель находится в третьей нормальной форме, за одним осознанным исключением. Столбцы `resident.balance` и `resident.loyalty_index` дублируют суммы журналов `balance_operation` и `loyalty_change`. Они хранятся для быстрой проверки условий подачи заявок и сортировки очереди; их согласованность с журналами обеспечивается триггерами (раздел 5).

Не являются избыточными столбцы `rent_charge.apartment_id` и `service_request.fee_paid`: первый фиксирует квартиру на момент начисления, которая может измениться после переселения, второй — пошлину, уплаченную в момент подачи заявки. Диаграмма в исходном размере приложена в файле `img/datalogical.pdf`.

Таблица 3 – Таблицы базы данных

Таблица	**Сущность**	**Первичный ключ**	**Внешние ключи**
`account`	Учётная запись	`id`	—
`apartment`	Квартира	`id`	—
`resident`	Жилец	`id`	`account_id (U)`, `apartment_id`
`invitation`	Код приглашения	`id`	`resident_id (U)`
`service`	Услуга	`id`	—
`service_request`	Заявка	`id`	`resident_id`, `service_id`, `target_apartment_id`, `status_changed_by`
`request_event`	Событие заявки	`id`	`request_id`, `actor_id`
`denunciation`	Донос	`id`	`author_id`, `suspect_id`, `reviewed_by`
`punishment`	Наказание	`id`	`resident_id`, `assigned_by`, `revoked_by_request_id`
`punishment_basis`	Связь M:N «Донос — Наказание»	(`punishment_id`, `denunciation_id`)	`punishment_id`, `denunciation_id`
`payment`	Платёж	`id`	`resident_id`
`rent_charge`	Начисление квартплаты	`id`	`resident_id`, `apartment_id`
`balance_operation`	Операция по балансу	`id`	`resident_id`, `payment_id (U)`, `rent_charge_id (U)`, `request_id`, `punishment_id (U)`
`loyalty_rule`	Правило ИЛ	`code`	—

Таблица	Сущность	Первичный ключ	Внешние ключи
loyalty_change	Изменение ИЛ	id	resident_id, rule_code, denunciation_id, punishment_id, rent_charge_id, request_id
dossier_note	Заметка досье	id	resident_id, author_id
notification	Уведомление	id	account_id

4. Реализация в PostgreSQL

Даталогическая модель реализована в виде набора SQL-скриптов (таблица 4). Скрипты не создают отдельную базу данных и схему: объекты создаются в текущей схеме (search_path). Это позволяет выполнять скрипты на сервере helios, где студенту доступна личная схема в базе studs, а команда CREATE DATABASE недоступна. Скрипт удаления удаляет все созданные объекты. Скрипты проверены на PostgreSQL 16 и используют возможности, доступные начиная с PostgreSQL 11 (процедуры, INCLUDE в индексах).

Таблица 4 – Скрипты базы данных

Файл	Назначение
install.sql	Создание базы целиком: последовательно выполняет скрипты 01–04.
00_drop.sql	Удаление всех объектов: представлений, таблиц, функций, процедур и типов. Повторный запуск безопасен.
01_create.sql	Перечислимые типы, таблицы, ограничения целостности, комментарии и справочные данные (услуги, правила ИЛ).
02_triggers.sql	Триггерные и вспомогательные функции, триггеры.
03_functions.sql	Функции и процедуры прецедентов, представления.
04_indexes.sql	Индексы.
05_fill.sql	Тестовые данные: 30 квартир, 24 жильца, управляющий, платежи, квартплата за два месяца, 13 заявок во всех статусах, 23 доноса, 15 наказаний.
06_fill_bulk.sql	Необязательные массовые данные для проверки индексов: 100 квартир, 300 жильцов, 10 000 заявок, 10 000 доносов (требование PRF002 SRS).
07_tests.sql	Проверка ограничений целостности; все изменения откатываются.

Порядок развёртывания на сервере helios:

```
cd sql
psql -h pg -d studs -f 00_drop.sql      # удалить предыдущую версию при( необходимости)
psql -h pg -d studs -f install.sql      # создать таблицы, триггеры, функции, индексы
psql -h pg -d studs -f 05_fill.sql      # заполнить тестовыми данными
psql -h pg -d studs -f 07_tests.sql     # проверить ограничения целостности
```

Тестовые данные создаются через функции прецедентов (раздел 6), поэтому журналы операций, история заявок, история ИЛ и уведомления согласованы между собой. Для входа в си-

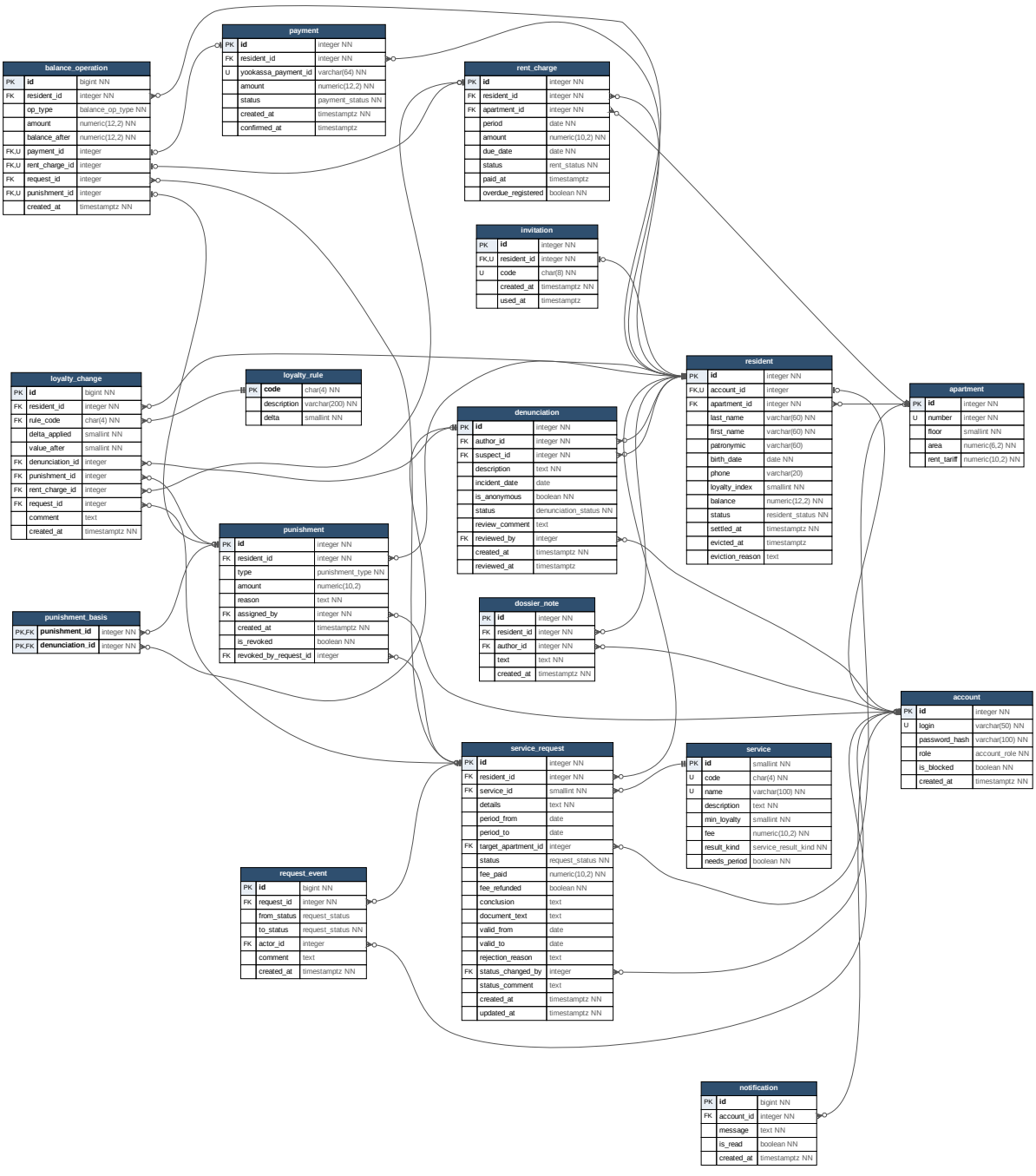


Рисунок 2 – Даталогическая модель базы данных (PK — первичный ключ, FK — внешний ключ, U — уникальное значение, NN — NOT NULL)

стему используются учётные записи `manager / manager123` (управляющий) и `gromov`, `belova` и др. `/ resident123` (жильцы); в базе хранятся bcrypt-хеши паролей.

5. Обеспечение целостности данных

5.1. Средства DDL

Таблица 5 – Ограничения целостности, заданные средствами DDL

Средство	Применение
Перечислимые типы	Допустимые значения ролей, статусов заявок, доносов, платежей, начислений, видов наказаний и типов операций.
`NOT NULL`	Обязательные атрибуты всех сущностей.
`PRIMARY KEY`, `FOREIGN KEY`	Ссылочная целостность. Для связей, удаление по которым недопустимо (жилец, заявка, донос), задано `ON DELETE RESTRICT`; для зависимых данных (код приглашения, заметки, уведомления) — `ON DELETE CASCADE`.
`UNIQUE`	Логин, код приглашения, номер квартиры, идентификатор платежа ЮKassa, одно начисление на квартиру и на жильца за период (переселившийся жилец не платит дважды), одна операция на платёж, начисление и штраф (REL002).
Частичные уникальные индексы	`uq_account_single_manager` — в системе один управляющий; `uq_resident_active_apartment` — в квартире проживает не более одного жильца.
`CHECK` на значения	ИЛ в диапазоне 0–100 (S010), положительные суммы платежей, неотрицательные пошлины и тарифы, формат логина, кода приглашения, кодов услуг и правил, начало периода начисления — первое число месяца.
`CHECK` на сочетания	Донос не может быть подан на себя (U026); статус «Выселен» тогда и только тогда, когда указана дата выселения; выполненная заявка имеет заключение, отклонённая — причину; сумма указывается только у штрафа; снять можно только предупреждение; знак суммы операции и ссылка на источник соответствуют типу операции.

5.2. Триггеры

Правила, которые нельзя выразить декларативно, реализованы триггерами (таблица 6). Триггеры обеспечивают выполнение бизнес-правил независимо от того, через какую функцию или напрямую изменяются данные.

Ключевое решение — *применение журналов*. Баланс и ИЛ жильца изменяются только вставкой записи в `balance_operation` или `loyalty_change`; триггер вставки вычисляет новое значение и обновляет строку жильца. Прямое изменение `resident.balance` или `resident.loyalty_index` отклоняется триггером `resident_guard`, который различает вызов из триггера журнала по глубине вложенности триггеров (`pg_trigger_depth()`). Таким образом текущие значения всегда совпадают с суммой записей журналов.

Таблица 6 – Триггеры

Триггер	Таблица	Событие	Назначение	SRS
balance_operation_apply	balance_operation	BEFORE INSERT	Применяет операцию к балансу жильца, запрещает уход в минус, кроме штрафов	S005, U017
loyalty_change_apply	loyalty_change	BEFORE INSERT	Изменяет ИЛ по правилу, ограничивает его диапазоном 0–100	S009, S010
*_immutable	balance_operation, loyalty_change, request_event	BEFORE UPDATE, DELETE	Запрещает изменение и удаление записей журналов	S013
resident_guard	resident	BEFORE INSERT, UPDATE	Задаёт начальные значения, запрещает дату рождения в будущем, изменение баланса и ИЛ в обход журналов и восстановление выселенного жильца	S008
resident_after_update	resident	AFTER UPDATE	Уведомляет об изменении ИЛ и о падении ИЛ ниже 10; блокирует учётную запись при выселении	S012, U030, C021
account_guard	account	BEFORE UPDATE	Запрещает смену роли	SEC002
invitation_guard	invitation	BEFORE UPDATE	Запрещает повторное использование кода	U002
request_before_insert	service_request	BEFORE INSERT	Проверяет статус жильца, ИЛ, задолженность, наличие периода, предупреждения (SV09) и свободной квартиры для переселения (SV07); фиксирует пошлину	U015, U017
request_after_insert	service_request	AFTER INSERT	Списывает пошлину, создаёт первую запись истории	U016, S006
request_before_update	service_request	BEFORE UPDATE	Проверяет переход по модели состояний заявки, запрещает менять заявителя, услугу, пошлину и результат	S006
request_after_update	service_request	AFTER UPDATE	Записывает историю, возвращает пошлину, уведомляет жильца	S006, U018, U030

Триггер	Таблица	Событие	Назначение	SRS
denunciation_before	denunciation	BEFORE INSERT, UPDATE	Проверяет, что участники проживают в доме и дата нарушения не в будущем; запрещает повторное рассмотрение и изменение содержания; проверяет роль рассматривающего	S007
denunciation_after_update	denunciation	AFTER UPDATE	Изменяет ИЛ неанонимного доносчика (RL01, RL02), уведомляет его	S009, U028
punishment_before	punishment	BEFORE INSERT, UPDATE	Проверяет роль управляющего и статус жильца, запрещает изменение назначенного наказания	C013
punishment_after_insert	punishment	AFTER INSERT	Применяет последствия: ИЛ (RL05, RL06), списание штрафа, выселение	S005, C021
punishment_basis_check	punishment_basis	BEFORE INSERT, UPDATE	Основанием может быть только подтверждённый донос на наказываемого жильца	C013
payment_before_update	payment	BEFORE UPDATE	Меняет статус только из PENDING, фиксирует время подтверждения	S003
payment_after_update	payment	AFTER UPDATE	Зачисляет средства ровно один раз	S003, REL002
rent_before_update	rent_charge	BEFORE UPDATE	Запрещает отмену оплаты и изменение параметров начисления	S001
rent_after_update	rent_charge	AFTER UPDATE	Применяет RL03 при оплате в срок и RL04 при регистрации просрочки	S002
dossier_note_check	dossier_note	BEFORE INSERT, UPDATE	Заметки добавляет только управляющий	C023

5.3. Проверка

Скрипт `07_tests.sql` выполняет 33 попытки нарушить ограничения целостности и 3 проверки согласованности данных. Все 36 проверок пройдены: каждая попытка нарушения отклонена с сообщением об ошибке, а баланс и ИЛ всех жильцов совпадают с журналами.

1. Донос на самого себя.
2. Прямое изменение баланса.
3. Прямое изменение ИЛ.
4. Создание жильца с ИЛ, отличным от 50.

5. Второй управляющий.
6. Заселение в занятую квартиру.
7. Повторная активация по коду.
8. Заявка при недостаточном ИЛ.
9. Заявка при задолженности.
10. Заявка выселенного жильца.
11. Разрешение без указания периода.
12. Недопустимый переход статуса заявки.
13. Отмена заявки, принятой в работу.
14. Изменение пошлины в заявке.
15. Действие управляющего от имени жильца.
16. Повторное рассмотрение доноса.
17. Изменение текста доноса.
18. Наказание по нерассмотренному доносу.
19. Наказание по доносу на другого жильца.
20. Штраф без суммы.
21. Оплата квартплаты при недостатке средств.
22. Изменение журнала операций по балансу.
23. Удаление записи истории ИЛ.
24. Удаление истории заявки.
25. Повторное изменение статуса платежа.
26. Отмена оплаты квартплаты.
27. Восстановление выселенного жильца.
28. Снятие предупреждения при их отсутствии.
29. Заметка в досье от жильца.
30. Отрицательная сумма платежа.
31. Переселение в занятую квартиру.
32. Дата нарушения в будущем.
33. Дата рождения в будущем.
34. Повторная обработка платежа не меняет баланс.
35. Баланс всех жильцов совпадает с журналом операций.
36. ИЛ всех жильцов совпадает с журналом изменений.

6. Функции и процедуры

Для каждого прецедента, изменяющего данные или требующего сложной выборки, реализована функция или процедура PL/pgSQL (таблица 7). Функции выполняют операцию прецедента одним вызовом и одной транзакцией (REL001 SRS), а проверку правил и побочные эффекты (списание средств, изменение ИЛ, запись истории, уведомления) берут на себя триггеры. Это исключает дублирование правил между функциями и серверной частью приложения.

Таблица 7 – Функции и процедуры прецедентов

Имя	Вид	Прецедент	Назначение
`settle_resident`	функция	UC13	Заселяет жильца в свободную квартиру, создаёт код приглашения.
`activate_account`	функция	UC01	Проверяет код, создаёт учётную запись жильца, отмечает код использованным.
`evict_resident`	процедура	UC14	Выселяет жильца: назначает наказание «выселение», последствия выполняют триггеры.

Имя	Вид	Прецедент	Назначение
register_payment	функция	UC04	Регистрирует платёж, созданный в ЮKassa.
apply_payment_status	функция	UC04	Применяет статус платежа из API ЮKassa; идемпотентна (REL002, COM003).
accrue_rent	процедура	S001	Начисляет квартплату за месяц жильцам, заселённым до конца месяца, без повторных начислений.
register_overdue_rent	функция	S002	Регистрирует просрочку неоплаченных начислений жильцов с активированной учётной записью; возвращает их число.
pay_rent	процедура	UC06	Оплачивает начисление квартплаты с баланса.
available_services	функция	UC03	Перечень услуг с признаком доступности по ИЛ жильца.
submit_request	функция	UC07	Подаёт заявку на услугу по коду услуги.
cancel_request	процедура	UC08	Отменяет заявку в статусе «Подана» с возвратом пошлины.
answer_clarification	процедура	UC09	Передаёт ответ жильца и возвращает заявку в работу.
request_queue	функция	UC17	Очередь необработанных заявок по убыванию ИЛ заявителей.
change_request_status	процедура	UC18, UC19	Принимает заявку в работу или запрашивает уточнение.
reject_request	процедура	UC20	Отклоняет заявку с причиной и решением о возврате пошлины.
complete_request	процедура	UC21	Завершает заявку; выполняет переселение (SV07) или снятие предупреждения (SV09).
submit_denunciation	функция	UC11	Регистрирует донос.
review_denunciation	процедура	UC22	Фиксирует решение по доносу.
assign_punishment	функция	UC23	Назначает наказание по массиву подтверждённых доносов (связь M:N) или без доноса.
add_dossier_note	функция	UC16	Добавляет заметку в досье.
resident_dossier	функция	UC15	Возвращает досье жильца одним JSON-документом.

Кроме функций созданы вспомогательная функция `reliability_status`, вычисляющая статус благонадёжности по ИЛ (S011), и представления:

- `v_denunciation_for_manager` — доносы для управляющего, у анонимных доносов автор скрыт (SEC004);
- `v_loyalty_history` — история ИЛ для жильца без ссылок на доносы (U007, SEC004);
- `v_apartment_occupancy` — квартиры с признаком занятости (C015).

Пример использования функций при подаче и обработке заявки:

```
SELECT submit_request(10, 'SV04', 'Приём␣родителей', '2026-11-14', '2026-11-15');  -- UC07
SELECT * FROM request_queue();                                                      -- UC17
CALL change_request_status(1, 5, 'IN_PROGRESS');                                    -- UC18
CALL complete_request(1, 5, 'Приём␣гостей␣разрешён.', 'РАЗРЕШЕНИЕ',                 -- UC21
                      '2026-11-14', '2026-11-15');
```

7. Индексы

7.1. Выбор индексов

PostgreSQL автоматически создаёт индексы только для первичных ключей и ограничений `UNIQUE`. Они обслуживают поиск по идентификатору, вход по логину, активацию по коду приглашения, поиск платежа по идентификатору ЮKassa и проверку уникальности начислений. Индексы по внешним ключам автоматически не создаются, поэтому выборки «все записи жильца» без дополнительных индексов выполняются полным просмотром таблицы.

Индексы выбраны по запросам, которые выполняются в прецедентах чаще всего или по быстрорастущим таблицам (таблица 8). Все индексы — B-деревья: условия отбора являются равенствами и диапазонами, а результат сортируется по дате. Составные индексы вида (*владелец*, *дата*) позволяют одновременно отобрать записи и получить их в нужном порядке без отдельной сортировки. Частичные индексы (I1, I4, I11, I13) построены только по «активной» части данных: необработанным заявкам и доносам, неоплаченным начислениям, непрочитанным уведомлениям. Их размер не растёт с накоплением истории. Начисления жильца (UC06) выбираются по индексу ограничения `UNIQUE (resident_id, period)`, поэтому отдельный индекс для них не создавался.

Таблица 8 – Индексы

№	Индекс	Определение	UC	Обоснование
I1	`idx_request_active`	`service_request (status, created_at) WHERE status IN (...)`	UC17	Очередь управляющего выбирает только необработанные заявки, доля которых мала. Частичный индекс содержит только их и не растёт с накоплением выполненных заявок.
I2	`idx_request_resident`	`service_request (resident_id, created_at DESC)`	UC10, UC15	Список заявок жильца, новые сначала; индекс обеспечивает и отбор, и сортировку.
I3	`idx_request_event_request`	`request_event (request_id, created_at)`	UC10	История заявки в хронологическом порядке; таблица истории самая быстрорастущая.

№	Индекс	Определение	UC	Обоснование
I4	idx_denunciation_pending	denunciation (created_at) WHERE status = 'SUBMITTED'	UC22	Очередь нерассмотренных доносов; частичный индекс.
I5	idx_denunciation_suspect	denunciation (suspect_id, created_at DESC)	UC15, UC23	Доносы на жильца в досье; выбор оснований наказания.
I6	idx_denunciation_author	denunciation (author_id, created_at DESC)	UC11	Список доносов, поданных жильцом (U028).
I7	idx_punishment_resident	punishment (resident_id, type) INCLUDE (is_revoked)	UC15, UC21	Наказания жильца; проверка наличия действующего предупреждения при подаче заявки SV09 (столбец is_revoked включён в индекс).
I8	idx_punishment_basis_denunciation	punishment_basis (denunciation_id)	UC15, UC23	Обратный поиск по связи M:N: наказания, назначенные по доносу. Прямой поиск обслуживает первичный ключ.
I9	idx_balance_operation_resident	balance_operation (resident_id, created_at DESC)	UC05	История операций по балансу жильца.
I10	idx_loyalty_change_resident	loyalty_change (resident_id, created_at DESC)	UC12	История ИЛ жильца.
I11	idx_rent_charge_overdue	rent_charge (due_date) WHERE status = 'UNPAID' AND NOT overdue_registered	S002	Ежедневный поиск просроченных начислений; частичный индекс содержит только неоплаченные начисления без зарегистрированной просрочки.
I12	idx_payment_resident	payment (resident_id, created_at DESC)	UC04, UC15	Платежи жильца.

№	Индекс	Определение	UC	Обоснование
I13	idx_notification_unread	notification (account_id, created_at DESC) WHERE NOT is_read	U030	Непрочитанные уведомления на главной странице каждого кабинета; таблица уведомлений самая большая.
I14	idx_dossier_note_resident	dossier_note (resident_id, created_at DESC)	UC15	Заметки досье жильца.

7.2. Проверка эффективности

Эффективность индексов проверена на массовых данных (скрипт `06_fill_bulk.sql`): 300 жильцов, 10 013 заявок, 28 454 записи истории заявок, 10 023 доноса, 30 600 уведомлений, 6 313 операций по балансу. Для каждого запроса выполнена команда `EXPLAIN ANALYZE` без индексов и с индексами; в таблице 9 приведено лучшее время из пяти запусков.

Таблица 9 – Планы и время выполнения запросов

№	Запрос	UC	Инд.	План без индекса	мс	План с индексом	мс	Уск.
Q1	Очередь заявок	UC17	I1	Seq Scan	3.56	Bitmap Index Scan	3.15	1.1 ×
Q2	Заявки жильца	UC10	I2	Seq Scan	1.48	Index Scan	0.13	11 ×
Q3	История заявки	UC10	I3	Seq Scan	2.23	Index Scan	0.06	38 ×
Q4	Очередь доносов	UC22	I4	Seq Scan	1.36	Index Scan	0.20	7 ×
Q5	Доносы на жильца	UC15	I5	Seq Scan	1.21	Bitmap Index Scan	0.53	2 ×
Q6	Доносы жильца	U028	I6	Seq Scan	1.78	Bitmap Index Scan	0.57	3 ×
Q7	Операции по балансу	UC05	I9	Seq Scan	0.94	Bitmap Index Scan	0.30	3 ×
Q8	История ИЛ	UC12	I10	Seq Scan	0.60	Bitmap Index Scan	0.29	2 ×
Q9	Просроченные начисления	S002	I11	Seq Scan	0.24	Bitmap Index Scan	0.04	6 ×
Q10	Непрочитанные уведомления	U030	I13	Seq Scan	3.66	Index Scan	0.13	29 ×

Без индексов каждый запрос выполняется полным просмотром таблицы (`Seq Scan`), и его время растёт линейно с объёмом данных. С индексами планировщик использует `Index Scan` или `Bitmap Index Scan`: время определяется числом отбираемых строк. Наибольший выигрыш (в 11–38 раз) получен для запросов, отбирающих несколько строк из больших таблиц: заявок жильца (Q2), истории заявки (Q3) и непрочитанных уведомлений (Q10). Для очереди заявок (Q1) выигрыш меньше, поскольку результат включает все необработанные заявки и сортируется по ИЛ заявителя из другой таблицы. Для начислений (Q9) индекс исключает просмотр всей истории начислений, которая растёт каждый месяц, при ежедневной проверке просрочки.

Индексы увеличивают стоимость вставки. Это допустимо: операции записи в системе единичны и выполняются пользователями, а чтение списков и очередей происходит при каждом открытии страницы кабинета.

8. Заключение

В ходе второго этапа построена ER-модель из 16 сущностей со связью «многие-ко-многим» «Донос — Наказание», на её основе получена даталогическая модель из 17 таблиц. Модель реализована в PostgreSQL; целостность данных обеспечена перечислимыми типами, ограничениями `NOT NULL`, `CHECK`, `UNIQUE`, внешними ключами и 23 триггерами. Реализованы скрипты создания, удаления и заполнения базы, 21 функция и процедура для прецедентов UC01–UC23, 14 индексов, полезность которых подтверждена измерениями. Функции будут использованы серверной частью приложения на следующем этапе.

Приложение А. Листинги скриптов

А.1. 01_create.sql

```
-- =============================================================
-- ИС Бдительность«». Этап 2.
-- 01_create.sql — создание таблиц, типов, ограничений и справочных данных.
-- Объекты создаются в текущей схеме (search_path), поэтому скрипт
-- можно выполнять на helios в личной схеме базы studs.
-- =============================================================

BEGIN;

-- -------------------------------------------------------------
-- Перечислимые типы
-- -------------------------------------------------------------
CREATE TYPE account_role        AS ENUM ('RESIDENT', 'MANAGER');
CREATE TYPE resident_status     AS ENUM ('ACTIVE', 'EVICTED');
CREATE TYPE service_result_kind AS ENUM ('WORK_MARK', 'DOCUMENT', 'TICKET', 'PERMIT', 'RELOCATION',
    'WARNING_REMOVAL');
CREATE TYPE request_status      AS ENUM ('SUBMITTED', 'IN_PROGRESS', 'NEEDS_CLARIFICATION', '
    COMPLETED', 'REJECTED', 'CANCELLED');
CREATE TYPE denunciation_status AS ENUM ('SUBMITTED', 'CONFIRMED', 'REJECTED', 'FALSE');
CREATE TYPE punishment_type     AS ENUM ('WARNING', 'FINE', 'EVICTION');
CREATE TYPE payment_status      AS ENUM ('PENDING', 'SUCCEEDED', 'CANCELED');
CREATE TYPE rent_status         AS ENUM ('UNPAID', 'PAID');
CREATE TYPE balance_op_type     AS ENUM ('TOPUP', 'RENT', 'FEE', 'FEE_REFUND', 'FINE');

-- -------------------------------------------------------------
-- Учётная запись
-- -------------------------------------------------------------
CREATE TABLE account (
    id             SERIAL PRIMARY KEY,
    login          VARCHAR(50)  NOT NULL UNIQUE
                   CHECK (login ~ '^[A-Za-z0-9_.]{3,50}$'),
    password_hash  VARCHAR(100) NOT NULL,
    role           account_role NOT NULL,
    is_blocked     BOOLEAN      NOT NULL DEFAULT FALSE,
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT now()
);
-- В системе может быть только один управляющий ограничение( 2.4 SRS)
CREATE UNIQUE INDEX uq_account_single_manager ON account (role) WHERE role = 'MANAGER';

-- -------------------------------------------------------------
-- Квартира
-- -------------------------------------------------------------
CREATE TABLE apartment (
    id           SERIAL PRIMARY KEY,
    number       INTEGER       NOT NULL UNIQUE CHECK (number > 0),
    floor        SMALLINT      NOT NULL CHECK (floor BETWEEN 1 AND 50),
    area         NUMERIC(6,2)  NOT NULL CHECK (area > 0),
    rent_tariff  NUMERIC(10,2) NOT NULL CHECK (rent_tariff >= 0)
);

-- -------------------------------------------------------------
-- Жилец
-- -------------------------------------------------------------
CREATE TABLE resident (
    id                 SERIAL PRIMARY KEY,
    account_id         INTEGER UNIQUE REFERENCES account (id) ON DELETE RESTRICT,
    apartment_id       INTEGER NOT NULL REFERENCES apartment (id) ON DELETE RESTRICT,
    last_name          VARCHAR(60)  NOT NULL CHECK (btrim(last_name)  <> ''),
    first_name         VARCHAR(60)  NOT NULL CHECK (btrim(first_name) <> ''),
    patronymic         VARCHAR(60),
    birth_date         DATE         NOT NULL CHECK (birth_date > DATE '1900-01-01'),
    phone              VARCHAR(20)  CHECK (phone ~ '^\+?[0-9\-␣]{5,20}$'),
    loyalty_index      SMALLINT     NOT NULL DEFAULT 50 CHECK (loyalty_index BETWEEN 0 AND 100),
    -- отрицательное значение баланса означает задолженность по штрафам
    balance            NUMERIC(12,2) NOT NULL DEFAULT 0,
    status             resident_status NOT NULL DEFAULT 'ACTIVE',
    settled_at         TIMESTAMPTZ  NOT NULL DEFAULT now(),
    evicted_at         TIMESTAMPTZ,
    eviction_reason    TEXT,
    CHECK ((status = 'EVICTED') = (evicted_at IS NOT NULL)),
    CHECK (status = 'ACTIVE' OR eviction_reason IS NOT NULL)
```

```
);
-- В квартире проживает не более одного жильца заселение( только в свободную квартиру)
CREATE UNIQUE INDEX uq_resident_active_apartment ON resident (apartment_id) WHERE status = 'ACTIVE';

-- ---------------------------------------------------------------------
-- Код приглашения
-- ---------------------------------------------------------------------
CREATE TABLE invitation (
    id           SERIAL PRIMARY KEY,
    resident_id  INTEGER     NOT NULL UNIQUE REFERENCES resident (id) ON DELETE CASCADE,
    code         CHAR(8)     NOT NULL UNIQUE CHECK (code ~ '^[A-Z0-9]{8}$'),
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    used_at      TIMESTAMPTZ,
    CHECK (used_at IS NULL OR used_at >= created_at)
);

-- ---------------------------------------------------------------------
-- Услуга фиксированный( справочник, таблица 2 SRS)
-- ---------------------------------------------------------------------
CREATE TABLE service (
    id            SMALLSERIAL PRIMARY KEY,
    code          CHAR(4)       NOT NULL UNIQUE CHECK (code ~ '^SV[0-9]{2}$'),
    name          VARCHAR(100)  NOT NULL UNIQUE,
    description   TEXT          NOT NULL,
    min_loyalty   SMALLINT      NOT NULL CHECK (min_loyalty BETWEEN 0 AND 100),
    fee           NUMERIC(10,2) NOT NULL CHECK (fee >= 0),
    result_kind   service_result_kind NOT NULL,
    needs_period  BOOLEAN       NOT NULL DEFAULT FALSE  -- требует указания дат разрешения()
);

-- ---------------------------------------------------------------------
-- Заявка на услугу
-- ---------------------------------------------------------------------
CREATE TABLE service_request (
    id                  SERIAL PRIMARY KEY,
    resident_id         INTEGER        NOT NULL REFERENCES resident (id) ON DELETE RESTRICT,
    service_id          SMALLINT       NOT NULL REFERENCES service (id) ON DELETE RESTRICT,
    details             TEXT           NOT NULL CHECK (btrim(details) <> ''),
    period_from         DATE,
    period_to           DATE,
    target_apartment_id INTEGER        REFERENCES apartment (id) ON DELETE RESTRICT,
    status              request_status NOT NULL DEFAULT 'SUBMITTED',
    fee_paid            NUMERIC(10,2)  NOT NULL CHECK (fee_paid >= 0),
    fee_refunded        BOOLEAN        NOT NULL DEFAULT FALSE,
    conclusion          TEXT,
    document_text       TEXT,
    valid_from          DATE,
    valid_to            DATE,
    rejection_reason    TEXT,
    -- кто и с каким комментарием изменил статус последним переносится( в историю триггером)
    status_changed_by   INTEGER        REFERENCES account (id),
    status_comment      TEXT,
    created_at          TIMESTAMPTZ    NOT NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ    NOT NULL DEFAULT now(),
    CHECK (period_from IS NULL OR period_to IS NULL OR period_from <= period_to),
    CHECK (valid_from  IS NULL OR valid_to  IS NULL OR valid_from  <= valid_to),
    CHECK (status <> 'COMPLETED' OR conclusion IS NOT NULL),
    CHECK (status <> 'REJECTED'  OR rejection_reason IS NOT NULL),
    CHECK (NOT fee_refunded OR status IN ('REJECTED', 'CANCELLED'))
);

-- ---------------------------------------------------------------------
-- Событие обработки заявки история( статусов, комментарии, уточнения)
-- ---------------------------------------------------------------------
CREATE TABLE request_event (
    id          BIGSERIAL PRIMARY KEY,
    request_id  INTEGER        NOT NULL REFERENCES service_request (id) ON DELETE RESTRICT,
    from_status request_status,
    to_status   request_status NOT NULL,
    actor_id    INTEGER        REFERENCES account (id),
    comment     TEXT,
    created_at  TIMESTAMPTZ    NOT NULL DEFAULT now(),
    CHECK (from_status IS DISTINCT FROM to_status)
);

-- ---------------------------------------------------------------------
```

```
-- Донос
-- ---------------------------------------------------------------------
CREATE TABLE denunciation (
    id             SERIAL PRIMARY KEY,
    author_id      INTEGER     NOT NULL REFERENCES resident (id) ON DELETE RESTRICT,
    suspect_id     INTEGER     NOT NULL REFERENCES resident (id) ON DELETE RESTRICT,
    description    TEXT        NOT NULL CHECK (length(btrim(description)) >= 10),
    incident_date  DATE,
    is_anonymous   BOOLEAN     NOT NULL DEFAULT FALSE,
    status         denunciation_status NOT NULL DEFAULT 'SUBMITTED',
    review_comment TEXT,
    reviewed_by    INTEGER     REFERENCES account (id),
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    reviewed_at    TIMESTAMPTZ,
    CHECK (author_id <> suspect_id),                              -- U026
    CHECK ((status = 'SUBMITTED') = (reviewed_at IS NULL)),
    CHECK (reviewed_at IS NULL OR reviewed_at >= created_at)
);

-- ---------------------------------------------------------------------
-- Наказание
-- ---------------------------------------------------------------------
CREATE TABLE punishment (
    id                  SERIAL PRIMARY KEY,
    resident_id         INTEGER         NOT NULL REFERENCES resident (id) ON DELETE RESTRICT,
    type                punishment_type NOT NULL,
    amount              NUMERIC(10,2),
    reason              TEXT            NOT NULL CHECK (btrim(reason) <> ''),
    assigned_by         INTEGER         NOT NULL REFERENCES account (id),
    created_at          TIMESTAMPTZ     NOT NULL DEFAULT now(),
    is_revoked          BOOLEAN         NOT NULL DEFAULT FALSE,
    revoked_by_request_id  INTEGER      REFERENCES service_request (id),
    CHECK ((type = 'FINE') = (amount IS NOT NULL)),
    CHECK (amount IS NULL OR amount > 0),
    CHECK (NOT is_revoked OR (type = 'WARNING' AND revoked_by_request_id IS NOT NULL))
);

-- ---------------------------------------------------------------------
-- Основание наказания: связь многие«комногим--» между наказанием и доносом.
-- Одно наказание может назначаться по нескольким доносам об одном нарушении,
-- по одному доносу может быть назначено несколько наказаний.
-- ---------------------------------------------------------------------
CREATE TABLE punishment_basis (
    punishment_id    INTEGER NOT NULL REFERENCES punishment (id)   ON DELETE CASCADE,
    denunciation_id  INTEGER NOT NULL REFERENCES denunciation (id) ON DELETE RESTRICT,
    PRIMARY KEY (punishment_id, denunciation_id)
);

-- ---------------------------------------------------------------------
-- Платёж ЮKassa
-- ---------------------------------------------------------------------
CREATE TABLE payment (
    id                  SERIAL PRIMARY KEY,
    resident_id         INTEGER        NOT NULL REFERENCES resident (id) ON DELETE RESTRICT,
    yookassa_payment_id VARCHAR(64)    NOT NULL UNIQUE,
    amount              NUMERIC(12,2)  NOT NULL CHECK (amount > 0),
    status              payment_status NOT NULL DEFAULT 'PENDING',
    created_at          TIMESTAMPTZ    NOT NULL DEFAULT now(),
    confirmed_at        TIMESTAMPTZ,
    CHECK ((status = 'SUCCEEDED') = (confirmed_at IS NOT NULL))
);

-- ---------------------------------------------------------------------
-- Начисление квартплаты
-- ---------------------------------------------------------------------
CREATE TABLE rent_charge (
    id                 SERIAL PRIMARY KEY,
    resident_id        INTEGER       NOT NULL REFERENCES resident (id)  ON DELETE RESTRICT,
    apartment_id       INTEGER       NOT NULL REFERENCES apartment (id) ON DELETE RESTRICT,
    period             DATE          NOT NULL CHECK (extract(day FROM period) = 1),
    amount             NUMERIC(10,2) NOT NULL CHECK (amount >= 0),
    due_date           DATE          NOT NULL,
    status             rent_status   NOT NULL DEFAULT 'UNPAID',
    paid_at            TIMESTAMPTZ,
    overdue_registered BOOLEAN       NOT NULL DEFAULT FALSE,
    UNIQUE (apartment_id, period),
```

```
    UNIQUE (resident_id, period),        -- переселившийся жилец не платит дважды за месяц
    CHECK (due_date > period),
    CHECK ((status = 'PAID') = (paid_at IS NOT NULL))
);

-- ---------------------------------------------------------------------
-- Операция по коммунальному балансу неизменяемый( журнал)
-- ---------------------------------------------------------------------
CREATE TABLE balance_operation (
    id              BIGSERIAL PRIMARY KEY,
    resident_id     INTEGER         NOT NULL REFERENCES resident (id) ON DELETE RESTRICT,
    op_type         balance_op_type NOT NULL,
    amount          NUMERIC(12,2)   NOT NULL CHECK (amount <> 0),
    balance_after   NUMERIC(12,2)   NOT NULL,
    payment_id      INTEGER UNIQUE  REFERENCES payment (id),
    rent_charge_id  INTEGER UNIQUE  REFERENCES rent_charge (id),
    request_id      INTEGER         REFERENCES service_request (id),
    punishment_id   INTEGER UNIQUE  REFERENCES punishment (id),
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    -- знак суммы и обязательная ссылка на источник зависят от типа операции
    CHECK (op_type <> 'TOPUP'      OR (amount > 0 AND payment_id     IS NOT NULL)),
    CHECK (op_type <> 'RENT'       OR (amount < 0 AND rent_charge_id IS NOT NULL)),
    CHECK (op_type <> 'FEE'        OR (amount < 0 AND request_id     IS NOT NULL)),
    CHECK (op_type <> 'FEE_REFUND' OR (amount > 0 AND request_id     IS NOT NULL)),
    CHECK (op_type <> 'FINE'       OR (amount < 0 AND punishment_id  IS NOT NULL))
);

-- ---------------------------------------------------------------------
-- Правило изменения ИЛ фиксированный( справочник, таблица 5 SRS)
-- ---------------------------------------------------------------------
CREATE TABLE loyalty_rule (
    code         CHAR(4)      PRIMARY KEY CHECK (code ~ '^RL[0-9]{2}$'),
    description  VARCHAR(200) NOT NULL,
    delta        SMALLINT     NOT NULL CHECK (delta <> 0)
);

-- ---------------------------------------------------------------------
-- Изменение ИЛ неизменяемый( журнал)
-- ---------------------------------------------------------------------
CREATE TABLE loyalty_change (
    id              BIGSERIAL PRIMARY KEY,
    resident_id     INTEGER     NOT NULL REFERENCES resident (id) ON DELETE RESTRICT,
    rule_code       CHAR(4)     NOT NULL REFERENCES loyalty_rule (code),
    delta_applied   SMALLINT    NOT NULL,          -- с учётом границ 0..100
    value_after     SMALLINT    NOT NULL CHECK (value_after BETWEEN 0 AND 100),
    denunciation_id INTEGER     REFERENCES denunciation (id),
    punishment_id   INTEGER     REFERENCES punishment (id),
    rent_charge_id  INTEGER     REFERENCES rent_charge (id),
    request_id      INTEGER     REFERENCES service_request (id),
    comment         TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ---------------------------------------------------------------------
-- Заметка в досье
-- ---------------------------------------------------------------------
CREATE TABLE dossier_note (
    id          SERIAL PRIMARY KEY,
    resident_id INTEGER     NOT NULL REFERENCES resident (id) ON DELETE CASCADE,
    author_id   INTEGER     NOT NULL REFERENCES account (id),
    text        TEXT        NOT NULL CHECK (btrim(text) <> ''),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ---------------------------------------------------------------------
-- Уведомление
-- ---------------------------------------------------------------------
CREATE TABLE notification (
    id          BIGSERIAL PRIMARY KEY,
    account_id  INTEGER     NOT NULL REFERENCES account (id) ON DELETE CASCADE,
    message     TEXT        NOT NULL,
    is_read     BOOLEAN     NOT NULL DEFAULT FALSE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ---------------------------------------------------------------------
```

```
-- Комментарии к таблицам
-- ---------------------------------------------------------------------
COMMENT ON TABLE account           IS 'Учётная запись пользователя жилец( или управляющий)';
COMMENT ON TABLE apartment         IS 'Квартира дома';
COMMENT ON TABLE resident          IS 'Жилец: анкета, ИЛ, коммунальный баланс';
COMMENT ON TABLE invitation        IS 'Одноразовый код приглашения для активации учётной записи';
COMMENT ON TABLE service           IS 'Услуга из фиксированного перечня';
COMMENT ON TABLE service_request   IS 'Заявка жильца на услугу';
COMMENT ON TABLE request_event     IS 'История обработки заявки';
COMMENT ON TABLE denunciation      IS 'Донос жильца на другого жильца';
COMMENT ON TABLE punishment        IS 'Наказание, назначенное жильцу';
COMMENT ON TABLE punishment_basis  IS 'Связь M:N наказаний и доносов, на основании которых они
    назначены';
COMMENT ON TABLE payment           IS 'Платёж пополнения баланса через ЮKassa';
COMMENT ON TABLE rent_charge       IS 'Ежемесячное начисление квартплаты';
COMMENT ON TABLE balance_operation IS 'Неизменяемый журнал операций по коммунальному балансу';
COMMENT ON TABLE loyalty_rule      IS 'Правило изменения индекса лояльности';
COMMENT ON TABLE loyalty_change    IS 'Неизменяемый журнал изменений индекса лояльности';
COMMENT ON TABLE dossier_note      IS 'Заметка управляющего в досье жильца';
COMMENT ON TABLE notification      IS 'Уведомление пользователю';
COMMENT ON COLUMN resident.balance IS 'Отрицательное значение — задолженность по штрафам';

-- ---------------------------------------------------------------------
-- Справочные данные, зафиксированные в системе (CON005)
-- ---------------------------------------------------------------------
INSERT INTO service (code, name, description, min_loyalty, fee, result_kind, needs_period) VALUES
 ('SV01', 'Бытовой ремонт',                          'Ремонт сантехники, электрики и иного
    оборудования квартиры.', 0,    0, 'WORK_MARK',       FALSE),
 ('SV02', 'Справка о проживании',                    'Справка, подтверждающая проживание в доме.',
                  20,  50, 'DOCUMENT',        FALSE),
 ('SV03', 'Продовольственный талон',                 'Талон на получение продуктов на указанный
    период.',          30,  50, 'TICKET',          TRUE),
 ('SV04', 'Разрешение на приём гостей',              'Разрешение принимать гостей в указанные даты.',
                  40, 100, 'PERMIT',          TRUE),
 ('SV05', 'Разрешение на выход в комендантский час', 'Разрешение находиться вне дома в комендантский
    час.',          50, 100, 'PERMIT',          TRUE),
 ('SV06', 'Замена мебели и бытовой техники',         'Замена мебели или бытовой техники в квартире.',
                  60, 300, 'WORK_MARK',       FALSE),
 ('SV07', 'Переселение в другую квартиру',           'Переселение в свободную квартиру дома.',
                  70, 300, 'RELOCATION',      FALSE),
 ('SV08', 'Разрешение на выезд из города',           'Разрешение покинуть город на указанный период.'
    ,             80, 300, 'PERMIT',          TRUE),
 ('SV09', 'Снятие предупреждения из досье',          'Снятие последнего действующего предупреждения.'
    ,             85, 500, 'WARNING_REMOVAL', FALSE);

INSERT INTO loyalty_rule (code, description, delta) VALUES
 ('RL01', 'Донос жильца подтверждён кроме( анонимного)',                  5),
 ('RL02', 'Донос жильца признан ложным кроме( анонимного)',             -10),
 ('RL03', 'Квартплата оплачена в течение 10 дней с даты начисления',      1),
 ('RL04', 'Квартплата не оплачена в течение 10 дней с даты начисления',  -5),
 ('RL05', 'Жильцу вынесено предупреждение',                              -10),
 ('RL06', 'Жильцу назначен штраф',                                       -15),
 ('RL07', 'Выполнена услуга SV09',                                        10);

COMMIT;
```

A.2. 02_triggers.sql

```
-- =====================================================================
-- ИС Бдительность«». Этап 2.
-- 02_triggers.sql — триггеры, обеспечивающие целостность данных.
-- =====================================================================

BEGIN;

-- ---------------------------------------------------------------------
-- Вспомогательные функции
-- ---------------------------------------------------------------------

-- Русское название статуса заявки для( уведомлений)
CREATE FUNCTION request_status_label(s request_status) RETURNS TEXT
LANGUAGE sql IMMUTABLE AS $$
    SELECT CASE s
```

```
        WHEN 'SUBMITTED'           THEN 'Подана'
        WHEN 'IN_PROGRESS'         THEN 'В␣работе'
        WHEN 'NEEDS_CLARIFICATION' THEN 'Требует␣уточнения'
        WHEN 'COMPLETED'           THEN 'Выполнена'
        WHEN 'REJECTED'            THEN 'Отклонена'
        WHEN 'CANCELLED'           THEN 'Отменена'
    END;
$$;

-- Русское название статуса доноса
CREATE FUNCTION denunciation_status_label(s denunciation_status) RETURNS TEXT
LANGUAGE sql IMMUTABLE AS $$
    SELECT CASE s
        WHEN 'SUBMITTED' THEN 'Подан'
        WHEN 'CONFIRMED' THEN 'Подтверждён'
        WHEN 'REJECTED'  THEN 'Отклонён'
        WHEN 'FALSE'     THEN 'Признан␣ложным'
    END;
$$;

-- Уведомление жильцу если( учётная запись активирована)
CREATE FUNCTION notify_resident(p_resident_id INTEGER, p_message TEXT) RETURNS VOID
LANGUAGE sql AS $$
    INSERT INTO notification (account_id, message)
    SELECT account_id, p_message FROM resident
    WHERE id = p_resident_id AND account_id IS NOT NULL;
$$;

-- Уведомление управляющему
CREATE FUNCTION notify_manager(p_message TEXT) RETURNS VOID
LANGUAGE sql AS $$
    INSERT INTO notification (account_id, message)
    SELECT id, p_message FROM account WHERE role = 'MANAGER';
$$;

-- Проверка, что учётная запись принадлежит управляющему
CREATE FUNCTION assert_manager(p_account_id INTEGER) RETURNS VOID
LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM account
                   WHERE id = p_account_id AND role = 'MANAGER' AND NOT is_blocked) THEN
        RAISE EXCEPTION 'Действие␣доступно␣только␣управляющему␣(account_id␣=␣%)', p_account_id;
    END IF;
END;
$$;

-- Проверка, что жилец проживает в доме не( выселен)
CREATE FUNCTION assert_active_resident(p_resident_id INTEGER) RETURNS VOID
LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM resident WHERE id = p_resident_id AND status = 'ACTIVE') THEN
        RAISE EXCEPTION 'Жилец␣%␣не␣проживает␣в␣доме', p_resident_id;
    END IF;
END;
$$;

-- ---------------------------------------------------------------------
-- 1. Неизменяемость журналов (S013): balance_operation, loyalty_change,
--    request_event
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_forbid_modification() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'Записи␣таблицы␣%␣не␣могут␣изменяться␣или␣удаляться', TG_TABLE_NAME;
END;
$$;

CREATE TRIGGER balance_operation_immutable BEFORE UPDATE OR DELETE ON balance_operation
    FOR EACH ROW EXECUTE FUNCTION trg_forbid_modification();
CREATE TRIGGER loyalty_change_immutable BEFORE UPDATE OR DELETE ON loyalty_change
    FOR EACH ROW EXECUTE FUNCTION trg_forbid_modification();
CREATE TRIGGER request_event_immutable BEFORE UPDATE OR DELETE ON request_event
    FOR EACH ROW EXECUTE FUNCTION trg_forbid_modification();

-- ---------------------------------------------------------------------
-- 2. Операция по балансу применяется к балансу жильца.
```

```
--    Уйти в минус задолженность() можно только изза- штрафа (S005, U017).
-- ------------------------------------------------------------------------
CREATE FUNCTION trg_balance_operation_apply() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE
    v_balance NUMERIC(12,2);
BEGIN
    SELECT balance INTO v_balance FROM resident WHERE id = NEW.resident_id FOR UPDATE;

    IF NEW.amount < 0 AND NEW.op_type <> 'FINE' AND v_balance + NEW.amount < 0 THEN
        RAISE EXCEPTION 'Недостаточно средств на балансе: % руб., требуется % руб.',
            v_balance, -NEW.amount;
    END IF;

    NEW.balance_after := v_balance + NEW.amount;
    UPDATE resident SET balance = NEW.balance_after WHERE id = NEW.resident_id;
    RETURN NEW;
END;
$$;

CREATE TRIGGER balance_operation_apply BEFORE INSERT ON balance_operation
    FOR EACH ROW EXECUTE FUNCTION trg_balance_operation_apply();

-- ------------------------------------------------------------------------
-- 3. Изменение ИЛ применяется к жильцу по правилу из справочника,
--    значение удерживается в диапазоне 0..100 (S009, S010).
-- ------------------------------------------------------------------------
CREATE FUNCTION trg_loyalty_change_apply() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE
    v_current SMALLINT;
    v_delta   SMALLINT;
BEGIN
    SELECT loyalty_index INTO v_current FROM resident WHERE id = NEW.resident_id FOR UPDATE;
    SELECT delta INTO v_delta FROM loyalty_rule WHERE code = NEW.rule_code;

    NEW.value_after   := LEAST(100, GREATEST(0, v_current + v_delta));
    NEW.delta_applied := NEW.value_after - v_current;
    UPDATE resident SET loyalty_index = NEW.value_after WHERE id = NEW.resident_id;
    RETURN NEW;
END;
$$;

CREATE TRIGGER loyalty_change_apply BEFORE INSERT ON loyalty_change
    FOR EACH ROW EXECUTE FUNCTION trg_loyalty_change_apply();

-- ------------------------------------------------------------------------
-- 4. Защита жильца: баланс и ИЛ меняются только через журналы,
--    новый жилец получает начальные значения (S008),
--    выселенного жильца нельзя вернуть.
-- ------------------------------------------------------------------------
CREATE FUNCTION trg_resident_guard() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        IF NEW.balance <> 0 OR NEW.loyalty_index <> 50 OR NEW.status <> 'ACTIVE' THEN
            RAISE EXCEPTION 'Новый жилец создаётся с нулевым балансом, ИЛ = 50 и статусом ACTIVE';
        END IF;
        IF NEW.birth_date > current_date THEN
            RAISE EXCEPTION 'Дата рождения не может быть в будущем';
        END IF;
        RETURN NEW;
    END IF;

    -- изменение баланса и ИЛ допускается только из триггеров журналов глубина( >= 2)
    IF (NEW.balance IS DISTINCT FROM OLD.balance
        OR NEW.loyalty_index IS DISTINCT FROM OLD.loyalty_index)
       AND pg_trigger_depth() < 2 THEN
        RAISE EXCEPTION 'Баланс и ИЛ изменяются только через balance_operation и loyalty_change';
    END IF;

    IF OLD.status = 'EVICTED' AND NEW.status = 'ACTIVE' THEN
        RAISE EXCEPTION 'Выселенный жилец не может быть восстановлен';
    END IF;

    IF OLD.status = 'EVICTED' AND NEW.apartment_id <> OLD.apartment_id THEN
```

```
        RAISE EXCEPTION 'Выселенного жильца нельзя переселить';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER resident_guard BEFORE INSERT OR UPDATE ON resident
    FOR EACH ROW EXECUTE FUNCTION trg_resident_guard();

-- ---------------------------------------------------------------------
-- 5. Последствия изменения жильца: уведомления об ИЛ (U030),
--    уведомление управляющего при ИЛ < 10 (S012),
--    блокировка учётной записи при выселении (U004, C021).
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_resident_after_update() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.loyalty_index <> OLD.loyalty_index THEN
        PERFORM notify_resident(NEW.id, format('Индекс лояльности изменён: %s → %s',
                                               OLD.loyalty_index, NEW.loyalty_index));
        IF NEW.loyalty_index < 10 AND OLD.loyalty_index >= 10 AND NEW.status = 'ACTIVE' THEN
            PERFORM notify_manager(format('ИЛ жильца %s %s кв(. %s) опустился до %s. Жилец подлежит
    выселению.',
                NEW.last_name, NEW.first_name,
                (SELECT number FROM apartment WHERE id = NEW.apartment_id), NEW.loyalty_index));
        END IF;
    END IF;

    IF NEW.status = 'EVICTED' AND OLD.status = 'ACTIVE' AND NEW.account_id IS NOT NULL THEN
        UPDATE account SET is_blocked = TRUE WHERE id = NEW.account_id;
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER resident_after_update AFTER UPDATE ON resident
    FOR EACH ROW EXECUTE FUNCTION trg_resident_after_update();

-- ---------------------------------------------------------------------
-- 6. Роль учётной записи неизменяема; код приглашения используется однократно
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_account_guard() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.role <> OLD.role THEN
        RAISE EXCEPTION 'Роль учётной записи не может быть изменена';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER account_guard BEFORE UPDATE ON account
    FOR EACH ROW EXECUTE FUNCTION trg_account_guard();

CREATE FUNCTION trg_invitation_guard() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.used_at IS NOT NULL THEN
        RAISE EXCEPTION 'Код приглашения уже использован';                -- U002
    END IF;
    IF NEW.code <> OLD.code OR NEW.resident_id <> OLD.resident_id THEN
        RAISE EXCEPTION 'Код приглашения не может быть изменён';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER invitation_guard BEFORE UPDATE ON invitation
    FOR EACH ROW EXECUTE FUNCTION trg_invitation_guard();

-- ---------------------------------------------------------------------
-- 7. Подача заявки: проверка условий (U015, U017), списание пошлины (U016),
--    первая запись истории (S006).
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_request_before_insert() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
```

```
DECLARE
    r resident%ROWTYPE;
    s service%ROWTYPE;
BEGIN
    SELECT * INTO r FROM resident WHERE id = NEW.resident_id;
    SELECT * INTO s FROM service  WHERE id = NEW.service_id;

    IF r.status <> 'ACTIVE' THEN
        RAISE EXCEPTION 'Выселенный␣жилец␣не␣может␣подавать␣заявки';
    END IF;
    IF r.loyalty_index < s.min_loyalty THEN
        RAISE EXCEPTION 'Услуга␣«%»␣требует␣ИЛ␣не␣менее␣%,␣текущий␣ИЛ␣%',
            s.name, s.min_loyalty, r.loyalty_index;
    END IF;
    IF r.balance < 0 THEN
        RAISE EXCEPTION 'Подача␣заявок␣невозможна␣при␣задолженности␣%␣руб.', -r.balance;
    END IF;
    IF s.needs_period AND (NEW.period_from IS NULL OR NEW.period_to IS NULL) THEN
        RAISE EXCEPTION 'Для␣услуги␣«%»␣необходимо␣указать␣период', s.name;
    END IF;
    IF s.result_kind = 'RELOCATION' THEN
        IF NEW.target_apartment_id IS NULL THEN
            RAISE EXCEPTION 'Для␣переселения␣необходимо␣указать␣квартиру';
        END IF;
        IF NEW.target_apartment_id = r.apartment_id THEN
            RAISE EXCEPTION 'Жилец␣уже␣проживает␣в␣указанной␣квартире';
        END IF;
        IF EXISTS (SELECT 1 FROM resident x
                    WHERE x.apartment_id = NEW.target_apartment_id AND x.status = 'ACTIVE') THEN
            RAISE EXCEPTION 'Квартира␣для␣переселения␣занята';
        END IF;
    END IF;
    IF s.result_kind = 'WARNING_REMOVAL' AND NOT EXISTS (
            SELECT 1 FROM punishment
            WHERE resident_id = r.id AND type = 'WARNING' AND NOT is_revoked) THEN
        RAISE EXCEPTION 'У␣жильца␣нет␣действующих␣предупреждений';
    END IF;

    NEW.status       := 'SUBMITTED';
    NEW.fee_paid     := s.fee;
    NEW.fee_refunded := FALSE;
    NEW.created_at   := now();
    NEW.updated_at   := now();
    RETURN NEW;
END;
$$;

CREATE TRIGGER request_before_insert BEFORE INSERT ON service_request
    FOR EACH ROW EXECUTE FUNCTION trg_request_before_insert();

CREATE FUNCTION trg_request_after_insert() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.fee_paid > 0 THEN
        INSERT INTO balance_operation (resident_id, op_type, amount, request_id)
        VALUES (NEW.resident_id, 'FEE', -NEW.fee_paid, NEW.id);
    END IF;
    INSERT INTO request_event (request_id, from_status, to_status, actor_id, comment)
    VALUES (NEW.id, NULL, NEW.status,
            (SELECT account_id FROM resident WHERE id = NEW.resident_id), 'Заявка␣подана');
    RETURN NULL;
END;
$$;

CREATE TRIGGER request_after_insert AFTER INSERT ON service_request
    FOR EACH ROW EXECUTE FUNCTION trg_request_after_insert();

-- ---------------------------------------------------------------------
-- 8. Изменение заявки: модель состояний таблица( 3 SRS, S006),
--    неизменяемость ключевых полей.
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_request_before_update() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.resident_id <> OLD.resident_id OR NEW.service_id <> OLD.service_id
       OR NEW.fee_paid <> OLD.fee_paid OR NEW.created_at <> OLD.created_at THEN
```

```
        RAISE EXCEPTION 'Заявитель, услуга, пошлина и дата подачи заявки не изменяются';
    END IF;
    IF OLD.fee_refunded AND NOT NEW.fee_refunded THEN
        RAISE EXCEPTION 'Возврат пошлины не может быть отменён';
    END IF;

    IF NEW.status <> OLD.status AND NOT (
           (OLD.status = 'SUBMITTED'           AND NEW.status IN ('IN_PROGRESS', 'REJECTED', '
    CANCELLED'))
        OR (OLD.status = 'IN_PROGRESS'         AND NEW.status IN ('NEEDS_CLARIFICATION', 'COMPLETED'
    , 'REJECTED'))
        OR (OLD.status = 'NEEDS_CLARIFICATION' AND NEW.status = 'IN_PROGRESS')) THEN
        RAISE EXCEPTION 'Недопустимый переход статуса заявки: % → %',
            request_status_label(OLD.status), request_status_label(NEW.status);
    END IF;

    IF OLD.status IN ('COMPLETED', 'REJECTED', 'CANCELLED')
       AND (NEW.conclusion IS DISTINCT FROM OLD.conclusion
            OR NEW.document_text IS DISTINCT FROM OLD.document_text
            OR NEW.rejection_reason IS DISTINCT FROM OLD.rejection_reason) THEN
        RAISE EXCEPTION 'Результат завершённой заявки не изменяется';
    END IF;

    -- комментарий предыдущего перехода не переносится в историю нового перехода
    IF NEW.status <> OLD.status AND NEW.status_comment IS NOT DISTINCT FROM OLD.status_comment THEN
        NEW.status_comment := NULL;
    END IF;

    NEW.updated_at := now();
    RETURN NEW;
END;
$$;

CREATE TRIGGER request_before_update BEFORE UPDATE ON service_request
    FOR EACH ROW EXECUTE FUNCTION trg_request_before_update();

-- ---------------------------------------------------------------------
-- 9. После изменения заявки: запись в историю (S006), возврат пошлины,
--    уведомление жильца (U030).
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_request_after_update() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.status <> OLD.status THEN
        INSERT INTO request_event (request_id, from_status, to_status, actor_id, comment)
        VALUES (NEW.id, OLD.status, NEW.status, NEW.status_changed_by, NEW.status_comment);

        PERFORM notify_resident(NEW.resident_id,
            format('Заявка № %s («%s»): статус «%s»', NEW.id,
                   (SELECT name FROM service WHERE id = NEW.service_id),
                   request_status_label(NEW.status)));
    END IF;

    IF NEW.fee_refunded AND NOT OLD.fee_refunded AND NEW.fee_paid > 0 THEN
        INSERT INTO balance_operation (resident_id, op_type, amount, request_id)
        VALUES (NEW.resident_id, 'FEE_REFUND', NEW.fee_paid, NEW.id);
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER request_after_update AFTER UPDATE ON service_request
    FOR EACH ROW EXECUTE FUNCTION trg_request_after_update();

-- ---------------------------------------------------------------------
-- 10. Донос: подача только проживающими жильцами на проживающих жильцов,
--     модель состояний таблица( 4 SRS, S007), неизменяемость содержания.
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_denunciation_before() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        PERFORM assert_active_resident(NEW.author_id);
        PERFORM assert_active_resident(NEW.suspect_id);
        IF NEW.incident_date > current_date THEN
            RAISE EXCEPTION 'Дата нарушения не может быть в будущем';
```

```
        END IF;
        NEW.status := 'SUBMITTED';
        NEW.reviewed_at := NULL;
        NEW.created_at := now();
        RETURN NEW;
    END IF;

    IF NEW.author_id <> OLD.author_id OR NEW.suspect_id <> OLD.suspect_id
       OR NEW.description <> OLD.description OR NEW.is_anonymous <> OLD.is_anonymous THEN
        RAISE EXCEPTION 'Содержание␣доноса␣не␣изменяется';
    END IF;

    IF NEW.status <> OLD.status THEN
        IF OLD.status <> 'SUBMITTED' THEN
            RAISE EXCEPTION 'Донос␣уже␣рассмотрен␣статус(␣«%»)', denunciation_status_label(OLD.status
    );
        END IF;
        PERFORM assert_manager(NEW.reviewed_by);
        NEW.reviewed_at := now();
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER denunciation_before BEFORE INSERT OR UPDATE ON denunciation
    FOR EACH ROW EXECUTE FUNCTION trg_denunciation_before();

-- ---------------------------------------------------------------------
-- 11. После рассмотрения доноса: изменение ИЛ доносчика по правилам
--     RL01 / RL02 кроме( анонимных доносов), уведомление доносчика.
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_denunciation_after_update() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.status <> OLD.status THEN
        IF NOT NEW.is_anonymous AND NEW.status IN ('CONFIRMED', 'FALSE') THEN
            INSERT INTO loyalty_change (resident_id, rule_code, denunciation_id, comment)
            VALUES (NEW.author_id,
                    CASE NEW.status WHEN 'CONFIRMED' THEN 'RL01' ELSE 'RL02' END,
                    NEW.id, format('Рассмотрен␣донос␣№␣%s', NEW.id));
        END IF;
        PERFORM notify_resident(NEW.author_id,
            format('Донос␣№␣%s␣рассмотрен:␣«%s»', NEW.id, denunciation_status_label(NEW.status)));
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER denunciation_after_update AFTER UPDATE ON denunciation
    FOR EACH ROW EXECUTE FUNCTION trg_denunciation_after_update();

-- ---------------------------------------------------------------------
-- 12. Наказание: назначается управляющим проживающему жильцу;
--     после назначения изменяется ИЛ (RL05, RL06), списывается штраф (S005),
--     при выселении изменяется статус жильца (C021).
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_punishment_before() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        PERFORM assert_manager(NEW.assigned_by);
        PERFORM assert_active_resident(NEW.resident_id);
        NEW.is_revoked := FALSE;
        NEW.revoked_by_request_id := NULL;
        NEW.created_at := now();
        RETURN NEW;
    END IF;

    IF NEW.resident_id <> OLD.resident_id OR NEW.type <> OLD.type
       OR NEW.amount IS DISTINCT FROM OLD.amount OR NEW.reason <> OLD.reason THEN
        RAISE EXCEPTION 'Назначенное␣наказание␣не␣изменяется';
    END IF;
    IF OLD.is_revoked AND NOT NEW.is_revoked THEN
        RAISE EXCEPTION 'Снятие␣предупреждения␣не␣может␣быть␣отменено';
    END IF;
    RETURN NEW;
```

```
END;
$$;

CREATE TRIGGER punishment_before BEFORE INSERT OR UPDATE ON punishment
    FOR EACH ROW EXECUTE FUNCTION trg_punishment_before();

CREATE FUNCTION trg_punishment_after_insert() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.type = 'WARNING' THEN
        INSERT INTO loyalty_change (resident_id, rule_code, punishment_id, comment)
        VALUES (NEW.resident_id, 'RL05', NEW.id, NEW.reason);
        PERFORM notify_resident(NEW.resident_id, format('Вам␣вынесено␣предупреждение:␣%s', NEW.reason
    ));

    ELSIF NEW.type = 'FINE' THEN
        INSERT INTO balance_operation (resident_id, op_type, amount, punishment_id)
        VALUES (NEW.resident_id, 'FINE', -NEW.amount, NEW.id);
        INSERT INTO loyalty_change (resident_id, rule_code, punishment_id, comment)
        VALUES (NEW.resident_id, 'RL06', NEW.id, NEW.reason);
        PERFORM notify_resident(NEW.resident_id,
            format('Вам␣назначен␣штраф␣%s␣руб.:␣%s', NEW.amount, NEW.reason));

    ELSE -- EVICTION
        UPDATE resident
        SET status = 'EVICTED', evicted_at = now(), eviction_reason = NEW.reason
        WHERE id = NEW.resident_id;
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER punishment_after_insert AFTER INSERT ON punishment
    FOR EACH ROW EXECUTE FUNCTION trg_punishment_after_insert();

-- ---------------------------------------------------------------------
-- 13. Основание наказания (M:N): донос должен быть подтверждён
--     и подан именно на наказываемого жильца.
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_punishment_basis_check() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM denunciation d JOIN punishment p ON p.id = NEW.punishment_id
        WHERE d.id = NEW.denunciation_id
          AND d.status = 'CONFIRMED'
          AND d.suspect_id = p.resident_id) THEN
        RAISE EXCEPTION 'Донос␣%␣не␣подтверждён␣или␣подан␣не␣на␣наказываемого␣жильца',
            NEW.denunciation_id;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER punishment_basis_check BEFORE INSERT OR UPDATE ON punishment_basis
    FOR EACH ROW EXECUTE FUNCTION trg_punishment_basis_check();

-- ---------------------------------------------------------------------
-- 14. Платёж: статус меняется только из PENDING; при успехе средства
--     зачисляются ровно один раз (S003, REL002).
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_payment_before_update() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.amount <> OLD.amount OR NEW.resident_id <> OLD.resident_id
       OR NEW.yookassa_payment_id <> OLD.yookassa_payment_id THEN
        RAISE EXCEPTION 'Параметры␣платежа␣не␣изменяются';
    END IF;
    IF NEW.status <> OLD.status THEN
        IF OLD.status <> 'PENDING' THEN
            RAISE EXCEPTION 'Платёж␣уже␣обработан␣статус(␣%)', OLD.status;
        END IF;
        IF NEW.status = 'SUCCEEDED' THEN
            NEW.confirmed_at := now();
        END IF;
    END IF;
```

```
    RETURN NEW;
END;
$$;

CREATE TRIGGER payment_before_update BEFORE UPDATE ON payment
    FOR EACH ROW EXECUTE FUNCTION trg_payment_before_update();

CREATE FUNCTION trg_payment_after_update() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.status = 'SUCCEEDED' AND OLD.status = 'PENDING' THEN
        INSERT INTO balance_operation (resident_id, op_type, amount, payment_id)
        VALUES (NEW.resident_id, 'TOPUP', NEW.amount, NEW.id);
        PERFORM notify_resident(NEW.resident_id,
            format('Баланс пополнен на %s руб.', NEW.amount));
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER payment_after_update AFTER UPDATE ON payment
    FOR EACH ROW EXECUTE FUNCTION trg_payment_after_update();

-- ---------------------------------------------------------------------
-- 15. Начисление квартплаты: оплата необратима; своевременная оплата
--     даёт RL03, регистрация просрочки — RL04 (S002).
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_rent_before_update() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.amount <> OLD.amount OR NEW.period <> OLD.period
       OR NEW.resident_id <> OLD.resident_id OR NEW.due_date <> OLD.due_date THEN
        RAISE EXCEPTION 'Параметры начисления не изменяются';
    END IF;
    IF OLD.status = 'PAID' AND NEW.status <> 'PAID' THEN
        RAISE EXCEPTION 'Оплата начисления не может быть отменена';
    END IF;
    IF OLD.overdue_registered AND NOT NEW.overdue_registered THEN
        RAISE EXCEPTION 'Регистрация просрочки не может быть отменена';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER rent_before_update BEFORE UPDATE ON rent_charge
    FOR EACH ROW EXECUTE FUNCTION trg_rent_before_update();

CREATE FUNCTION trg_rent_after_update() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.status = 'PAID' AND OLD.status = 'UNPAID' AND NEW.paid_at::date <= NEW.due_date THEN
        INSERT INTO loyalty_change (resident_id, rule_code, rent_charge_id, comment)
        VALUES (NEW.resident_id, 'RL03', NEW.id,
                format('Квартплата за %s оплачена в срок', to_char(NEW.period, 'MM.YYYY')));
    END IF;
    IF NEW.overdue_registered AND NOT OLD.overdue_registered THEN
        INSERT INTO loyalty_change (resident_id, rule_code, rent_charge_id, comment)
        VALUES (NEW.resident_id, 'RL04', NEW.id,
                format('Просрочена квартплата за %s', to_char(NEW.period, 'MM.YYYY')));
        PERFORM notify_resident(NEW.resident_id,
            format('Просрочена оплата квартплаты за %s', to_char(NEW.period, 'MM.YYYY')));
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER rent_after_update AFTER UPDATE ON rent_charge
    FOR EACH ROW EXECUTE FUNCTION trg_rent_after_update();

-- ---------------------------------------------------------------------
-- 16. Заметки досье добавляет только управляющий
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_dossier_note_check() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    PERFORM assert_manager(NEW.author_id);
```

```
    RETURN NEW;
END;
$$;

CREATE TRIGGER dossier_note_check BEFORE INSERT OR UPDATE ON dossier_note
    FOR EACH ROW EXECUTE FUNCTION trg_dossier_note_check();

COMMIT;
```

A.3. 03_functions.sql

```
-- ============================================================================
-- ИС Бдительность«». Этап 2.
-- 03_functions.sql — функции и процедуры PL/pgSQL для реализации прецедентов,
-- а также представления, скрывающие авторов доносов (SEC004).
-- ============================================================================

BEGIN;

-- ----------------------------------------------------------------------------
-- Статус благонадёжности (S011, таблица 6 SRS) — вычисляется по ИЛ
-- ----------------------------------------------------------------------------
CREATE FUNCTION reliability_status(p_loyalty INTEGER) RETURNS TEXT
LANGUAGE sql IMMUTABLE AS $$
    SELECT CASE WHEN p_loyalty >= 80 THEN 'Образцовый'
                WHEN p_loyalty >= 50 THEN 'Благонадёжен'
                WHEN p_loyalty >= 20 THEN 'Под␣подозрением'
                ELSE 'Неблагонадёжен' END;
$$;

-- ----------------------------------------------------------------------------
-- Представления
-- ----------------------------------------------------------------------------

-- Доносы для управляющего: автор анонимного доноса не раскрывается (SEC004)
CREATE VIEW v_denunciation_for_manager AS
SELECT d.id,
       CASE WHEN d.is_anonymous THEN NULL ELSE d.author_id END AS author_id,
       d.suspect_id, d.description, d.incident_date, d.is_anonymous,
       d.status, d.review_comment, d.created_at, d.reviewed_at
FROM denunciation d;

-- История ИЛ для жильца: ссылки на доносы не выдаются (U007, SEC004)
CREATE VIEW v_loyalty_history AS
SELECT lc.id, lc.resident_id, lc.rule_code, lr.description AS rule_description,
       lc.delta_applied, lc.value_after, lc.created_at
FROM loyalty_change lc JOIN loyalty_rule lr ON lr.code = lc.rule_code;

-- Квартиры с признаком занятости (C015)
CREATE VIEW v_apartment_occupancy AS
SELECT a.id, a.number, a.floor, a.area, a.rent_tariff,
       r.id AS resident_id,
       r.last_name || '␣' || r.first_name AS resident_name,
       (r.id IS NOT NULL) AS is_occupied
FROM apartment a
LEFT JOIN resident r ON r.apartment_id = a.id AND r.status = 'ACTIVE';

-- ----------------------------------------------------------------------------
-- Жилой фонд и учётные записи
-- ----------------------------------------------------------------------------

-- UC13. Заселить жильца. Возвращает идентификатор жильца и код приглашения.
CREATE FUNCTION settle_resident(
    p_manager_id   INTEGER,
    p_apartment_id INTEGER,
    p_last_name    VARCHAR,
    p_first_name   VARCHAR,
    p_patronymic   VARCHAR,
    p_birth_date   DATE,
    p_phone        VARCHAR DEFAULT NULL)
RETURNS TABLE (resident_id INTEGER, invite_code CHAR(8))
LANGUAGE plpgsql AS $$
DECLARE
```

```
    v_resident INTEGER;
    v_code     CHAR(8);
BEGIN
    PERFORM assert_manager(p_manager_id);

    -- занятость квартиры дополнительно контролирует уникальный индекс
    IF EXISTS (SELECT 1 FROM resident r WHERE r.apartment_id = p_apartment_id AND r.status = 'ACTIVE
    ') THEN
        RAISE EXCEPTION 'Квартира␣%␣занята', p_apartment_id;
    END IF;

    INSERT INTO resident (apartment_id, last_name, first_name, patronymic,
                          birth_date, phone)
    VALUES (p_apartment_id, p_last_name, p_first_name, p_patronymic,
            p_birth_date, p_phone)
    RETURNING id INTO v_resident;

    LOOP   -- генерация уникального кода
        v_code := upper(substr(md5(random()::text || clock_timestamp()::text), 1, 8));
        EXIT WHEN NOT EXISTS (SELECT 1 FROM invitation i WHERE i.code = v_code);
    END LOOP;

    INSERT INTO invitation (resident_id, code) VALUES (v_resident, v_code);
    RETURN QUERY SELECT v_resident, v_code;
END;
$$;

-- UC01. Активировать учётную запись по коду приглашения.
-- Пароль хешируется приложением (bcrypt, SEC001), в БД передаётся хеш.
CREATE FUNCTION activate_account(p_code CHAR(8), p_login VARCHAR, p_password_hash VARCHAR)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_inv     invitation%ROWTYPE;
    v_account INTEGER;
BEGIN
    SELECT * INTO v_inv FROM invitation WHERE code = upper(p_code) FOR UPDATE;
    IF NOT FOUND OR v_inv.used_at IS NOT NULL THEN
        RAISE EXCEPTION 'Код␣приглашения␣неверен␣или␣уже␣использован';
    END IF;
    PERFORM assert_active_resident(v_inv.resident_id);

    INSERT INTO account (login, password_hash, role)
    VALUES (p_login, p_password_hash, 'RESIDENT')
    RETURNING id INTO v_account;

    UPDATE resident   SET account_id = v_account WHERE id = v_inv.resident_id;
    UPDATE invitation SET used_at = now()        WHERE id = v_inv.id;
    RETURN v_account;
END;
$$;

-- UC14. Выселить жильца по( решению управляющего или после уведомления S012).
CREATE PROCEDURE evict_resident(p_manager_id INTEGER, p_resident_id INTEGER, p_reason TEXT)
LANGUAGE plpgsql AS $$
BEGIN
    PERFORM assert_manager(p_manager_id);
    PERFORM assert_active_resident(p_resident_id);
    INSERT INTO punishment (resident_id, type, reason, assigned_by)
    VALUES (p_resident_id, 'EVICTION', p_reason, p_manager_id);
    -- статус жильца, блокировку учётной записи и освобождение квартиры
    -- выполняют триггеры punishment_after_insert и resident_after_update
END;
$$;

-- ====================================================================
-- Баланс и квартплата
-- ====================================================================

-- UC04 шаг( 2). Зарегистрировать платёж, созданный в ЮKassa.
CREATE FUNCTION register_payment(p_resident_id INTEGER, p_yookassa_id VARCHAR, p_amount NUMERIC)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_id INTEGER;
BEGIN
```

```
    PERFORM assert_active_resident(p_resident_id);
    INSERT INTO payment (resident_id, yookassa_payment_id, amount)
    VALUES (p_resident_id, p_yookassa_id, p_amount)
    RETURNING id INTO v_id;
    RETURN v_id;
END;
$$;

-- UC04 шаги( –56). Применить статус платежа, полученный из API ЮKassa (COM003).
-- Идемпотентна: повторный вызов не изменяет баланс (REL002).
-- Возвращает TRUE, если средства зачислены именно этим вызовом.
CREATE FUNCTION apply_payment_status(p_yookassa_id VARCHAR, p_status payment_status)
RETURNS BOOLEAN
LANGUAGE plpgsql AS $$
DECLARE
    v_pay payment%ROWTYPE;
BEGIN
    SELECT * INTO v_pay FROM payment WHERE yookassa_payment_id = p_yookassa_id FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Платёж % не найден', p_yookassa_id;
    END IF;
    IF v_pay.status <> 'PENDING' OR p_status = 'PENDING' THEN
        RETURN FALSE;
    END IF;
    UPDATE payment SET status = p_status WHERE id = v_pay.id;   -- зачисление выполняет триггер
    RETURN p_status = 'SUCCEEDED';
END;
$$;

-- S001. Начислить квартплату за месяц по всем занятым квартирам.
-- Начисляется жильцам, заселённым до конца месяца; повторный запуск
-- за тот же месяц не создаёт дублей.
CREATE PROCEDURE accrue_rent(p_period DATE)
LANGUAGE plpgsql AS $$
DECLARE
    v_period DATE := date_trunc('month', p_period)::date;
BEGIN
    INSERT INTO rent_charge (resident_id, apartment_id, period, amount, due_date)
    SELECT r.id, a.id, v_period, a.rent_tariff, v_period + 10
    FROM resident r JOIN apartment a ON a.id = r.apartment_id
    WHERE r.status = 'ACTIVE'
      AND r.settled_at < v_period + INTERVAL '1 month'
    ON CONFLICT DO NOTHING;
END;
$$;

-- S002. Зарегистрировать просрочку по неоплаченным начислениям (RL04 — триггер).
-- Просрочка регистрируется только жильцам, активировавшим учётную запись:
-- до активации жилец не может оплатить начисление, а его ИЛ остаётся
-- начальным (S008).
CREATE FUNCTION register_overdue_rent(p_today DATE DEFAULT current_date)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_count INTEGER;
BEGIN
    UPDATE rent_charge rc SET overdue_registered = TRUE
    FROM resident r
    WHERE r.id = rc.resident_id AND r.status = 'ACTIVE' AND r.account_id IS NOT NULL
      AND rc.status = 'UNPAID' AND NOT rc.overdue_registered
      AND rc.due_date < p_today;
    GET DIAGNOSTICS v_count = ROW_COUNT;
    RETURN v_count;
END;
$$;

-- UC06. Оплатить начисление квартплаты с баланса.
CREATE PROCEDURE pay_rent(p_resident_id INTEGER, p_charge_id INTEGER)
LANGUAGE plpgsql AS $$
DECLARE
    v_rc rent_charge%ROWTYPE;
BEGIN
    SELECT * INTO v_rc FROM rent_charge WHERE id = p_charge_id FOR UPDATE;
    IF NOT FOUND OR v_rc.resident_id <> p_resident_id THEN
        RAISE EXCEPTION 'Начисление % не принадлежит жильцу %', p_charge_id, p_resident_id;
    END IF;
```

```
    IF v_rc.status = 'PAID' THEN
        RAISE EXCEPTION 'Начисление уже оплачено';
    END IF;
    IF v_rc.amount > 0 THEN    -- при недостатке средств триггер прервёт операцию
        INSERT INTO balance_operation (resident_id, op_type, amount, rent_charge_id)
        VALUES (p_resident_id, 'RENT', -v_rc.amount, v_rc.id);
    END IF;
    UPDATE rent_charge SET status = 'PAID', paid_at = now() WHERE id = v_rc.id;
END;
$$;

-- ====================================================================
-- Заявки на услуги
-- ====================================================================

-- UC03. Перечень услуг с признаком доступности для жильца (U013, U014).
CREATE FUNCTION available_services(p_resident_id INTEGER)
RETURNS TABLE (service_id SMALLINT, code CHAR(4), name VARCHAR, description TEXT,
               min_loyalty SMALLINT, fee NUMERIC, result_kind service_result_kind,
               needs_period BOOLEAN, is_available BOOLEAN)
LANGUAGE sql STABLE AS $$
    SELECT s.id, s.code, s.name, s.description, s.min_loyalty, s.fee,
           s.result_kind, s.needs_period,
           r.status = 'ACTIVE' AND r.loyalty_index >= s.min_loyalty
    FROM service s CROSS JOIN resident r
    WHERE r.id = p_resident_id
    ORDER BY s.min_loyalty, s.code;
$$;

-- UC07. Подать заявку. Проверки и списание пошлины выполняют триггеры.
CREATE FUNCTION submit_request(
    p_resident_id         INTEGER,
    p_service_code        CHAR(4),
    p_details             TEXT,
    p_period_from         DATE    DEFAULT NULL,
    p_period_to           DATE    DEFAULT NULL,
    p_target_apartment_id INTEGER DEFAULT NULL)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_service SMALLINT;
    v_id      INTEGER;
BEGIN
    SELECT id INTO v_service FROM service WHERE code = p_service_code;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Услуга % не существует', p_service_code;
    END IF;
    INSERT INTO service_request (resident_id, service_id, details,
                                 period_from, period_to, target_apartment_id, fee_paid)
    VALUES (p_resident_id, v_service, p_details,
            p_period_from, p_period_to, p_target_apartment_id, 0)
    RETURNING id INTO v_id;
    RETURN v_id;
END;
$$;

-- UC08. Отменить заявку в статусе Подана«» с возвратом пошлины (U018).
CREATE PROCEDURE cancel_request(p_resident_id INTEGER, p_request_id INTEGER)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE service_request
    SET status = 'CANCELLED', fee_refunded = (fee_paid > 0),
        status_changed_by = (SELECT account_id FROM resident WHERE id = p_resident_id),
        status_comment = 'Отменена жильцом'
    WHERE id = p_request_id AND resident_id = p_resident_id AND status = 'SUBMITTED';
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Заявку % нельзя отменить', p_request_id;
    END IF;
END;
$$;

-- UC09. Ответить на запрос уточнения (U021).
CREATE PROCEDURE answer_clarification(p_resident_id INTEGER, p_request_id INTEGER, p_answer TEXT)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE service_request
```

```
    SET status = 'IN_PROGRESS',
        status_changed_by = (SELECT account_id FROM resident WHERE id = p_resident_id),
        status_comment = p_answer
    WHERE id = p_request_id AND resident_id = p_resident_id AND status = 'NEEDS_CLARIFICATION';
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Заявка␣%␣не␣ожидает␣уточнения', p_request_id;
    END IF;
END;
$$;

-- UC17. Очередь заявок для управляющего: по убыванию ИЛ, затем по дате (C001).
CREATE FUNCTION request_queue()
RETURNS TABLE (request_id INTEGER, created_at TIMESTAMPTZ, status request_status,
               service_name VARCHAR, resident_id INTEGER, resident_name TEXT,
               apartment_number INTEGER, loyalty_index SMALLINT)
LANGUAGE sql STABLE AS $$
    SELECT sr.id, sr.created_at, sr.status, s.name,
           r.id, r.last_name || '␣' || r.first_name, a.number, r.loyalty_index
    FROM service_request sr
    JOIN resident  r ON r.id = sr.resident_id
    JOIN service   s ON s.id = sr.service_id
    JOIN apartment a ON a.id = r.apartment_id
    WHERE sr.status IN ('SUBMITTED', 'IN_PROGRESS', 'NEEDS_CLARIFICATION')
      AND r.status = 'ACTIVE'
    ORDER BY r.loyalty_index DESC, sr.created_at, sr.id;
$$;

-- Общая процедура смены статуса заявки управляющим (UC18, UC19)
CREATE PROCEDURE change_request_status(p_manager_id INTEGER, p_request_id INTEGER,
                                       p_status request_status, p_comment TEXT DEFAULT NULL)
LANGUAGE plpgsql AS $$
BEGIN
    PERFORM assert_manager(p_manager_id);
    IF p_status NOT IN ('IN_PROGRESS', 'NEEDS_CLARIFICATION') THEN
        RAISE EXCEPTION 'Для␣завершения␣и␣отклонения␣используйте␣complete_request␣и␣reject_request';
    END IF;
    IF p_status = 'NEEDS_CLARIFICATION' AND coalesce(btrim(p_comment), '') = '' THEN
        RAISE EXCEPTION 'Запрос␣уточнения␣должен␣содержать␣вопрос';
    END IF;
    UPDATE service_request
    SET status = p_status, status_changed_by = p_manager_id, status_comment = p_comment
    WHERE id = p_request_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Заявка␣%␣не␣найдена', p_request_id;
    END IF;
END;
$$;

-- UC20. Отклонить заявку с указанием причины и решением о возврате пошлины.
CREATE PROCEDURE reject_request(p_manager_id INTEGER, p_request_id INTEGER,
                                p_reason TEXT, p_refund BOOLEAN)
LANGUAGE plpgsql AS $$
BEGIN
    PERFORM assert_manager(p_manager_id);
    UPDATE service_request
    SET status = 'REJECTED', rejection_reason = p_reason,
        fee_refunded = p_refund AND fee_paid > 0,
        status_changed_by = p_manager_id, status_comment = p_reason
    WHERE id = p_request_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Заявка␣%␣не␣найдена', p_request_id;
    END IF;
END;
$$;

-- UC21. Завершить заявку с оформлением результата и применением
-- последствий услуги: переселение (SV07), снятие предупреждения (SV09).
CREATE PROCEDURE complete_request(p_manager_id INTEGER, p_request_id INTEGER,
                                  p_conclusion TEXT,
                                  p_document_text TEXT DEFAULT NULL,
                                  p_valid_from DATE DEFAULT NULL,
                                  p_valid_to   DATE DEFAULT NULL)
LANGUAGE plpgsql AS $$
DECLARE
    v_req  service_request%ROWTYPE;
    v_kind service_result_kind;
```

```
    v_warn INTEGER;
BEGIN
    PERFORM assert_manager(p_manager_id);
    SELECT * INTO v_req FROM service_request WHERE id = p_request_id FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Заявка␣%␣не␣найдена', p_request_id;
    END IF;
    SELECT result_kind INTO v_kind FROM service WHERE id = v_req.service_id;

    IF v_kind IN ('PERMIT', 'TICKET') AND (p_valid_from IS NULL OR p_valid_to IS NULL) THEN
        RAISE EXCEPTION 'Для␣разрешения␣или␣талона␣необходимо␣указать␣срок␣действия';
    END IF;

    UPDATE service_request
    SET status = 'COMPLETED', conclusion = p_conclusion, document_text = p_document_text,
        valid_from = p_valid_from, valid_to = p_valid_to,
        status_changed_by = p_manager_id, status_comment = p_conclusion
    WHERE id = p_request_id;

    IF v_kind = 'RELOCATION' THEN
        IF EXISTS (SELECT 1 FROM resident x
                   WHERE x.apartment_id = v_req.target_apartment_id AND x.status = 'ACTIVE') THEN
            RAISE EXCEPTION 'Квартира␣для␣переселения␣занята';
        END IF;
        UPDATE resident SET apartment_id = v_req.target_apartment_id WHERE id = v_req.resident_id;

    ELSIF v_kind = 'WARNING_REMOVAL' THEN
        SELECT id INTO v_warn FROM punishment
        WHERE resident_id = v_req.resident_id AND type = 'WARNING' AND NOT is_revoked
        ORDER BY created_at DESC, id DESC LIMIT 1
        FOR UPDATE;
        IF v_warn IS NULL THEN
            RAISE EXCEPTION 'У␣жильца␣нет␣действующих␣предупреждений';
        END IF;
        UPDATE punishment SET is_revoked = TRUE, revoked_by_request_id = p_request_id
        WHERE id = v_warn;
        INSERT INTO loyalty_change (resident_id, rule_code, request_id, comment)
        VALUES (v_req.resident_id, 'RL07', p_request_id, 'Снято␣предупреждение');
    END IF;
END;
$$;

-- ======================================================================
-- Доносы и наказания
-- ======================================================================

-- UC11. Подать донос (-U025U027).
CREATE FUNCTION submit_denunciation(p_author_id INTEGER, p_suspect_id INTEGER,
                                    p_description TEXT, p_incident_date DATE,
                                    p_is_anonymous BOOLEAN)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_id INTEGER;
BEGIN
    IF p_author_id = p_suspect_id THEN
        RAISE EXCEPTION 'Нельзя␣подать␣донос␣на␣самого␣себя';
    END IF;
    INSERT INTO denunciation (author_id, suspect_id, description, incident_date, is_anonymous)
    VALUES (p_author_id, p_suspect_id, p_description, p_incident_date, p_is_anonymous)
    RETURNING id INTO v_id;
    RETURN v_id;
END;
$$;

-- UC22. Рассмотреть донос: CONFIRMED / REJECTED / FALSE.
-- Изменение ИЛ доносчика и уведомление выполняют триггеры.
CREATE PROCEDURE review_denunciation(p_manager_id INTEGER, p_denunciation_id INTEGER,
                                     p_decision denunciation_status, p_comment TEXT DEFAULT NULL)
LANGUAGE plpgsql AS $$
BEGIN
    IF p_decision = 'SUBMITTED' THEN
        RAISE EXCEPTION 'Решение␣по␣доносу␣должно␣быть␣CONFIRMED,␣REJECTED␣или␣FALSE';
    END IF;
    UPDATE denunciation
    SET status = p_decision, reviewed_by = p_manager_id, review_comment = p_comment
```

```
    WHERE id = p_denunciation_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Донос␣%␣не␣найден', p_denunciation_id;
    END IF;
END;
$$;

-- UC23. Назначить наказание по одному или нескольким подтверждённым доносам
-- пустой( массив — наказание без доноса, C014).
CREATE FUNCTION assign_punishment(p_manager_id INTEGER, p_resident_id INTEGER,
                                  p_type punishment_type, p_reason TEXT,
                                  p_amount NUMERIC DEFAULT NULL,
                                  p_denunciation_ids INTEGER[] DEFAULT '{}')
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_id INTEGER;
BEGIN
    INSERT INTO punishment (resident_id, type, amount, reason, assigned_by)
    VALUES (p_resident_id, p_type, CASE WHEN p_type = 'FINE' THEN p_amount END,
            p_reason, p_manager_id)
    RETURNING id INTO v_id;

    INSERT INTO punishment_basis (punishment_id, denunciation_id)
    SELECT v_id, unnest(p_denunciation_ids);
    RETURN v_id;
END;
$$;

-- ========================================================================
-- Досье
-- ========================================================================

-- UC16. Добавить заметку в досье.
CREATE FUNCTION add_dossier_note(p_manager_id INTEGER, p_resident_id INTEGER, p_text TEXT)
RETURNS INTEGER
LANGUAGE sql AS $$
    INSERT INTO dossier_note (resident_id, author_id, text)
    VALUES (p_resident_id, p_manager_id, p_text)
    RETURNING id;
$$;

-- UC15. Досье жильца в виде JSONдокумента-. Авторы анонимных доносов не выдаются.
CREATE FUNCTION resident_dossier(p_resident_id INTEGER)
RETURNS JSONB
LANGUAGE sql STABLE AS $$
    SELECT jsonb_build_object(
        'resident', (SELECT to_jsonb(x) FROM (
            SELECT r.id, r.last_name, r.first_name, r.patronymic, r.birth_date,
                   r.phone, a.number AS apartment, r.loyalty_index,
                   reliability_status(r.loyalty_index) AS reliability_status, r.balance, r.status, r
    .settled_at, r.evicted_at
            FROM resident r JOIN apartment a ON a.id = r.apartment_id
            WHERE r.id = p_resident_id) x),
        'denunciations_against', coalesce((SELECT jsonb_agg(to_jsonb(d) ORDER BY d.created_at DESC,
    d.id DESC)
            FROM v_denunciation_for_manager d WHERE d.suspect_id = p_resident_id), '[]'),
        'denunciations_filed', coalesce((SELECT jsonb_agg(jsonb_build_object(
                'id', d.id, 'suspect_id', d.suspect_id, 'status', d.status,
                'created_at', d.created_at) ORDER BY d.created_at DESC, d.id DESC)
            FROM denunciation d WHERE d.author_id = p_resident_id AND NOT d.is_anonymous), '[]'),
        'punishments', coalesce((SELECT jsonb_agg(jsonb_build_object(
                'id', p.id, 'type', p.type, 'amount', p.amount, 'reason', p.reason,
                'created_at', p.created_at, 'is_revoked', p.is_revoked,
                'denunciations', (SELECT coalesce(jsonb_agg(pb.denunciation_id), '[]')
                                  FROM punishment_basis pb WHERE pb.punishment_id = p.id))
                ORDER BY p.created_at DESC, p.id DESC)
            FROM punishment p WHERE p.resident_id = p_resident_id), '[]'),
        'loyalty_history', coalesce((SELECT jsonb_agg(to_jsonb(h) ORDER BY h.created_at DESC, h.id
    DESC)
            FROM v_loyalty_history h WHERE h.resident_id = p_resident_id), '[]'),
        'requests', coalesce((SELECT jsonb_agg(jsonb_build_object(
                'id', sr.id, 'service', s.name, 'status', sr.status,
                'created_at', sr.created_at) ORDER BY sr.created_at DESC, sr.id DESC)
            FROM service_request sr JOIN service s ON s.id = sr.service_id
            WHERE sr.resident_id = p_resident_id), '[]'),
```

```
        'payments', coalesce((SELECT jsonb_agg(jsonb_build_object(
                'id', bo.id, 'type', bo.op_type, 'amount', bo.amount,
                'balance_after', bo.balance_after, 'created_at', bo.created_at)
                ORDER BY bo.created_at DESC, bo.id DESC)
            FROM balance_operation bo WHERE bo.resident_id = p_resident_id), '[]'),
        'notes', coalesce((SELECT jsonb_agg(jsonb_build_object(
                'id', n.id, 'text', n.text, 'created_at', n.created_at)
                ORDER BY n.created_at DESC, n.id DESC)
            FROM dossier_note n WHERE n.resident_id = p_resident_id), '[]')
    );
$$;

COMMIT;
```

A.4. 04_indexes.sql

```
-- ======================================================================
-- ИС Бдительность«». Этап 2.
-- 04_indexes.sql — индексы для запросов, выполняемых в прецедентах.
-- Индексы первичных ключей и ограничений UNIQUE создаются автоматически
-- (01_create.sql); здесь созданы индексы по внешним ключам и условиям
-- отбора, которые PostgreSQL автоматически не создаёт.
-- Начисления жильца (UC06) выбираются по уникальному индексу
-- ограничения UNIQUE (resident_id, period), отдельный индекс не нужен.
-- ======================================================================

BEGIN;

-- I1. UC17 Посмотреть« список заявок»: очередь необработанных заявок.
-- Частичный индекс содержит только активные заявки, доля которых мала.
CREATE INDEX idx_request_active ON service_request (status, created_at)
    WHERE status IN ('SUBMITTED', 'IN_PROGRESS', 'NEEDS_CLARIFICATION');

-- I2. UC10, U019 Мои« заявки», UC15 досье: заявки конкретного жильца, новые сначала.
CREATE INDEX idx_request_resident ON service_request (resident_id, created_at DESC);

-- I3. UC10, U020: история обработки заявки.
CREATE INDEX idx_request_event_request ON request_event (request_id, created_at);

-- I4. UC22 Рассмотреть« донос»: очередь нерассмотренных доносов.
CREATE INDEX idx_denunciation_pending ON denunciation (created_at)
    WHERE status = 'SUBMITTED';

-- I5. UC15 Посмотреть« досье»: доносы на жильца; проверка оснований наказания.
CREATE INDEX idx_denunciation_suspect ON denunciation (suspect_id, created_at DESC);

-- I6. U028 Мои« доносы»: доносы, поданные жильцом.
CREATE INDEX idx_denunciation_author ON denunciation (author_id, created_at DESC);

-- I7. UC23, UC15: наказания жильца; поиск действующего предупреждения (SV09).
CREATE INDEX idx_punishment_resident ON punishment (resident_id, type)
    INCLUDE (is_revoked);

-- I8. Связь M:N: обратный поиск наказаний по доносу
-- прямой( поиск по punishment_id обслуживает первичный ключ).
CREATE INDEX idx_punishment_basis_denunciation ON punishment_basis (denunciation_id);

-- I9. UC05 Просмотреть« баланс и историю операций».
CREATE INDEX idx_balance_operation_resident ON balance_operation (resident_id, created_at DESC);

-- I10. UC12 Просмотреть« историю ИЛ».
CREATE INDEX idx_loyalty_change_resident ON loyalty_change (resident_id, created_at DESC);

-- I11. S002: ежедневный поиск просроченных начислений.
CREATE INDEX idx_rent_charge_overdue ON rent_charge (due_date)
    WHERE status = 'UNPAID' AND NOT overdue_registered;

-- I12. UC05, UC15: платежи жильца.
CREATE INDEX idx_payment_resident ON payment (resident_id, created_at DESC);

-- I13. U030: непрочитанные уведомления на главной странице кабинета.
CREATE INDEX idx_notification_unread ON notification (account_id, created_at DESC)
    WHERE NOT is_read;
```

```
-- I14. UC15: заметки досье.
CREATE INDEX idx_dossier_note_resident ON dossier_note (resident_id, created_at DESC);

COMMIT;
```

Скрипты `00_drop.sql`, `05_fill.sql`, `06_fill_bulk.sql`, `07_tests.sql` и `install.sql` приложены к отчёту отдельными файлами.